INTERFACES GRAFICAS

REPORTE JUEGO-UNO

INTEGRANTES:
-Víctor Sebastián Guevara Rangel
-Ximena Ovalle Peña
-Núñez Mejía Ángel Enrique
-Joaquín Emmanuel Zapata Martínez

Índice

1. Panorama general 5

1.1 ¿Qué es el proyecto? 5

1.2 Archivos del repositorio 6

1.3 Cómo se ejecuta 7

1.4 Flujo completo de una partida 7

2. Base de datos (scriptuno.sql) 8

2.1 Creación de la base (líneas 1–3) 8

2.2 Tabla carta (líneas 9–26) 8

2.3 Tabla jugador (líneas 32–36) 9

2.4 Tabla partida (líneas 42–51) 9

2.5 Tabla partida_jugador (líneas 57–69) 10

2.6 Tabla jugada: el log (líneas 82–111) 10

2.7 Vistas (líneas 118–143) 12

2.8 Datos: las 60 cartas (líneas 151–213) 13

2.9 Diagrama entidad-relación 14

3. API en Python (db.py y main.py) 16

3.1 db.py: la conexión a MySQL 16

3.2 main.py: imports y creación de la app (líneas 1–20) 17

3.3 Conexión y manejo de errores (líneas 27–55) 17

3.4 Modelos de entrada con Pydantic (líneas 62–88) 19

3.5 Endpoints 20

3.6 Códigos HTTP que puede devolver la API 24

4. C#: punto de entrada, modelos y comunicación con la API 26

4.1 Conceptos de C# que aparecen en todo el proyecto 26

4.2 Program.cs: el punto de entrada 26

4.3 Carta.cs: el modelo de una carta 27

4.4 Jugador.cs: el modelo de un jugador 28

4.5 BaseDatos.cs: el cliente de la API 28

5. Form1.cs: la clase Ventana (lógica del juego) 35

5.1 Constantes y variables de estado (líneas 15–70) 35

5.2 Constructor Ventana() (líneas 72–93) 37

5.3 AplicarEstilo (líneas 98–160) 38

5.4 CrearIndicadorTurno (líneas 165–179) 40

5.5 EmpezarPartida (líneas 184–242) 40

5.6 JugarCarta: la regla principal (líneas 244–303) 42

5.7 EfectoCartaInvalida (líneas 305–344) 44

5.8 botonPasar_Click (líneas 350–368) 45

5.9 PedirColor (líneas 373–383) 46

5.10 AplicarEfecto (líneas 388–413) 46

5.11 Siguiente: el cálculo del turno (líneas 418–422) 47

5.12 DarCartas (líneas 429–462) 47

5.13 Fin de la partida (líneas 464–506) 48

5.14 Barajar, Repartir, carta inicial y mazo (líneas 511–563) 49

5.15 Dibujar las manos (líneas 565–624) 50

5.16 Descarte, turno y título (líneas 629–703) 52

5.17 Imágenes de las cartas (líneas 709–768) 54

5.18 Acomodo de la mesa (líneas 777–913) 55

5.19 RobarCarta (líneas 915–933) 59

5.20 Decir UNO (líneas 943–1019) 59

5.21 Reglas implementadas vs. reglas oficiales 61

6. Las demás ventanas 62

6.1 FormInicio.cs: pantalla de inicio 62

6.2 FormNombres.cs: capturar los nombres 67

6.3 FormElegirColor: elegir color del comodín 69

6.4 FormGanador.cs: pantalla del ganador 71

7. Archivos del diseñador y de configuración 75

7.1 Form1.Designer.cs 75

7.2 FormInicio.Designer.cs . . . 76
7.3 Configuración del proyecto . . . 76
8. Historial de commits: qué hizo cada quien . . . 78
8.1 Resumen por integrante . . . 78
8.2 Diagrama de ramas . . . 79
8.3 Todos los commits en orden . . . 82
8.4 Problemas que se ven en el historial . . . 85
9. Guía de estudio . . . 86
9.1 Dónde está cada cosa importante . . . 86
9.2 Preguntas que te pueden hacer (con respuesta) . . . 87
9.3 Glosario rápido . . . 88

1. Panorama general

1.1 ¿Qué es el proyecto?

En este documento se explica el código del proyecto archivo por archivo. Los números de línea que se mencionan corresponden a la versión final que está en la rama `master` del repositorio `github.com/CrAsI-fr/Juego-UNO`.

Es el juego de cartas UNO para 4 jugadores en una sola computadora: una persona controla a los cuatro jugadores y todas las cartas se ven boca arriba. Cada movimiento del juego se guarda en una base de datos MySQL para tener un historial completo de las partidas.

El proyecto tiene tres capas que se comunican en cadena. El profesor pidió que el programa en C# no se conecte directamente a MySQL; por eso existe una API intermedia en Python:

Capa	Tecnología	Archivos	Qué hace
1. Interfaz y reglas	C# WinForms (.NET Framework 4.7.2)	`Juego-UNO/*.cs`	Dibuja la mesa, reparte, valida jugadas, maneja turnos. Todo el estado del juego vive aquí, en memoria.
2. API	Python + FastAPI + uvicorn	`main.py`, `db.py`	Recibe peticiones HTTP con JSON desde C#, las valida y las traduce a consultas SQL.
3. Base de datos	MySQL 8	`scriptuno.sql`	Guarda el catálogo de cartas, jugadores, partidas, quién jugó cada partida y el log de cada movimiento.

El recorrido de un dato es siempre el mismo:

Paso	Ejemplo: el jugador tira un "Rojo 5"
C# (`Form1.cs`)	`JugarCarta` valida la jugada y llama a `BaseDatos.RegistrarJugada(...)`.
C# (`BaseDatos.cs`)	Convierte los datos a JSON y manda `POST http://127.0.0.1:8000/partidas/7/jugadas`.
Python (`main.py`)	FastAPI valida el JSON con el modelo `JugadaEntrada` y ejecuta un `INSERT` en la tabla `jugada`.
MySQL	Revisa llaves foráneas y `check`; si todo está bien guarda la fila.
Regreso	La API responde `{"id_jugada": ..., "num_jugada": ...}`; si hubo error, C# lanza `ErrorBaseDatos` y muestra un mensaje.

La base de datos no decide nada del juego. Solo registra. Las manos de los jugadores, el mazo, el turno y la dirección viven en variables de la clase `Ventana` (archivo `Form1.cs`). Esta es una de las ideas más importantes para explicar el proyecto.

1.2 Archivos del repositorio

Archivo	Líneas	Para qué sirve
`scriptuno.sql`	213	Crea la base `unobd`: 5 tablas, 2 vistas y las 60 cartas.
`db.py`	20	Función `get_connection()` que abre la conexión a MySQL leyendo `.env`.
`main.py`	233	La API FastAPI: 10 endpoints.
`librerias.txt`	19	Librerías de Python con su versión (se instalan con `pip install -r`).
`.env.ejemplo`	4	Plantilla de los datos de conexión (el `.env` real no se sube a Git).
`.gitignore` / `.gitattributes`	365 / –	Qué archivos ignora Git (bin, obj, .venv, .env...) y cómo maneja finales de línea.
`Juego-UNO.slnx`	3	Archivo de solución de Visual Studio; apunta al `.csproj`.
`Juego-UNO/Program.cs`	22	Punto de entrada: abre `FormInicio`.
`Juego-UNO/FormInicio.cs`	260	Pantalla de inicio + clase `BotonUno` (botón dibujado a mano).
`Juego-UNO/Form1.cs`	1021	Clase `Ventana`: toda la lógica del juego y el acomodo de la mesa.
`Juego-UNO/Form1.Designer.cs`	234	Controles creados con el diseñador (paneles de mano, mazo, descarte, etiquetas, botón Pasar).
`Juego-UNO/BaseDatos.cs`	252	Cliente HTTP que habla con la API, y la excepción `ErrorBaseDatos`.
`Juego-UNO/Carta.cs`	37	Modelo de una carta.
`Juego-UNO/Jugador.cs`	20	Modelo de un jugador (con su mano).
`Juego-UNO/FormNombres.cs`	113	Ventana para escribir los 4 nombres.
`Juego-UNO/FormElegirColor.cs` (+ Designer)	57 + 123	Ventana para escoger color al tirar un comodín.
`Juego-UNO/FormGanador.cs`	191	Pantalla del ganador con confeti animado.
`Juego-UNO/Cartas/*.png`	50 imágenes	Una imagen por tipo de carta (las repetidas usan la misma imagen).
`Juego-UNO/Properties/*`	–	Archivos generados por Visual Studio (versión, recursos, configuración).
`Juego-UNO/*.resx`	–	Recursos incrustados: imagen de fondo, icono, imágenes de colores.

Archivo	Líneas	Para qué sirve
`Juego-UNO.csproj`, `packages.config`, `App.config`	–	Configuración del proyecto C# y paquetes NuGet.

1.3 Cómo se ejecuta

- MySQL: ejecutar `scriptuno.sql` en MySQL Workbench. Borra y vuelve a crear la base `unobd`.
- Entorno de Python: en la carpeta raíz, `py -m venv .venv`, activar con `.venv\Scripts\Activate.ps1` e instalar con `pip install -r librerias.txt`.
- Archivo .env: copiar `.env.ejemplo` a `.env` y poner la contraseña real de MySQL.
- Levantar la API: `uvicorn main:app --reload`. La documentación automática queda en `http://127.0.0.1:8000/docs`.
- Juego: abrir `Juego-UNO.slnx` en Visual Studio y ejecutar (F5).

Nota: `--reload` hace que uvicorn reinicie la API cada vez que se guarda un archivo `.py`. `main:app` significa "el objeto `app` que está en el archivo `main.py`".

1.4 Flujo completo de una partida

- `Program.Main` abre FormInicio. Cada 2 segundos revisa si la API responde ("● Conectado" / "● Error de conexión").
- Clic en JUGAR → se oculta el inicio y se abre la mesa (`Ventana`). Al mostrarse, llama a `EmpezarPartida()`.
- `EmpezarPartida` abre FormNombres, da de alta o busca a los 4 jugadores en la API, pide las 60 cartas, las baraja, reparte 7 a cada quien, voltea una carta inicial de número y registra todo en una sola petición (`POST /partidas`).
- Los jugadores juegan con doble clic en una carta. Si la carta no se puede jugar, se pinta de rojo un momento. Si es comodín se abre FormElegirColor. Clic en el mazo = robar. "Pasar" aparece después de robar.
- Al quedar con 1 carta aparece el botón ¡UNO! con cuenta regresiva de 4 s. Si no se presiona, castigo de 2 cartas.
- Al quedar con 0 cartas se registra el ganador (`PUT /partidas/{id}/terminar`), aparece FormGanador con confeti y al cerrar se vuelve a FormInicio.

2. Base de datos (scriptuno.sql)

2.1 Creación de la base (líneas 1–3)

Fragmento de scriptuno.sql (líneas 1 a 3)

```
1  drop database if exists unobd;
2  create database unobd;
3  use unobd;
```

- Línea 1: `drop database if exists unobd;` borra la base si ya existía. Por eso el script se puede correr varias veces, pero borra todos los datos.
- Línea 2: crea la base vacía y línea 3 la selecciona para que las siguientes instrucciones trabajen dentro de ella.

2.2 Tabla carta (líneas 9–26)

Es el catálogo del mazo. Cada fila es una carta física; si hay dos "Rojo bloqueo", son dos filas con distinto `id_carta`.

Fragmento de scriptuno.sql (líneas 9 a 26)

```
 9  create table `unobd`.`carta` (
10      `id_carta` int not null auto_increment,
11      `tipo_carta` enum('ordinaria', 'especial') not null,
12      `color_carta` enum('rojo', 'amarillo', 'azul', 'verde') null,
13      `efecto` enum('cambiar_color', 'cambiar_direccion', 'bloquear', '+2', '+4') null,
14      `num_carta` int null,
15      primary key (`id_carta`),
16      constraint chk_num_carta check (num_carta <= 9 and num_carta >= 0),
17      -- ordinaria: color + numero, sin efecto
18      -- especial comodin (cambiar_color, +4): sin color ni numero
19      -- especial de color (bloquear, +2, cambiar_direccion): color, sin numero
20      constraint chk_carta_consistente check (
21          (tipo_carta = 'ordinaria'
22              and color_carta is not null and num_carta is not null and efecto is null)
23          or (tipo_carta = 'especial' and efecto in ('cambiar_color', '+4')
24              and color_carta is null and num_carta is null)
25          or (tipo_carta = 'especial' and efecto in ('bloquear', '+2', 'cambiar_direccion')
26              and color_carta is not null and num_carta is null)));
```

- Línea 10: `id_carta` es la llave primaria y `auto_increment` (MySQL pone 1, 2, 3...).
- Línea 11: `tipo_carta` es un `enum`: solo acepta `'ordinaria'` (de número) o `'especial'`.
- Línea 12: `color_carta` puede ser `null` porque los comodines no tienen color.
- Línea 13: `efecto` solo para especiales: cambiar color, reversa (`cambiar_direccion`), bloqueo, +2, +4.
- Línea 14: `num_carta` solo para ordinarias.
- Línea 16: `chk_num_carta`: el número debe estar entre 0 y 9.

- Líneas 20 a 26: `chk_carta_consistente`: impide filas sin sentido. Una carta es válida solo si cumple uno de tres casos:

Caso	Color	Número	Efecto
Ordinaria	obligatorio	obligatorio	debe ser `null`
Comodín (`cambiar_color`, `+4`)	debe ser `null`	debe ser `null`	obligatorio
Especial de color (`bloquear`, `+2`, `cambiar_direccion`)	obligatorio	debe ser `null`	obligatorio

Un `CHECK` es una regla que MySQL revisa en cada `INSERT`/`UPDATE`. Si no se cumple, MySQL rechaza la fila con el error 3819. La API convierte ese error en una respuesta 400 (ver `main.py` línea 44).

2.3 Tabla jugador (líneas 32–36)

Fragmento de scriptuno.sql (líneas 32 a 36)

```
32  create table `unobd`.`jugador` (
33      `id_jugador` int not null auto_increment,
34      `nombre_jug` varchar(30) not null,
35      primary key (`id_jugador`),
36      unique key `uq_nombre_jug` (`nombre_jug`));
```

- Solo guarda datos permanentes: id y nombre. La mano no se guarda aquí (vive en memoria del programa C#).
- Línea 36: `unique key uq_nombre_jug`: no puede haber dos jugadores con el mismo nombre. Gracias a esto la API puede "buscar o crear" por nombre y un mismo jugador acumula historial entre partidas.
- Línea 34: `varchar(30)`: por eso `FormNombres.cs` limita las cajas de texto a 30 caracteres (`LargoMaximo = 30`, línea 14).

2.4 Tabla partida (líneas 42–51)

Fragmento de scriptuno.sql (líneas 42 a 51)

```
42  create table `unobd`.`partida` (
43      `id_partida` int not null auto_increment,
44      `fecha_inicio` timestamp not null default current_timestamp,
45      `fecha_fin` timestamp null default null,
46      `id_ganador` int null default null,
47      primary key (`id_partida`),
48      key `id_ganador` (`id_ganador`),
49      constraint `fk_partida_ganador`
50          foreign key (`id_ganador`)
51          references `jugador` (`id_jugador`));
```

- Línea 44: `fecha_inicio` se llena sola con la hora actual (`default current_timestamp`).
- Línea 45: `fecha_fin` es `null` mientras la partida está en curso.

- Línea 46: `id_ganador` es `null` si la partida sigue o si se abandonó (se cerró el programa sin ganador).
- Líneas 49 a 51: llave foránea: `id_ganador` debe existir en `jugador.id_jugador`.

fecha_fin	id_ganador	Significado
`null`	`null`	Partida en curso
con fecha	con id	Terminó con ganador
con fecha	`null`	Abandonada (se cerró la mesa o se empezó otra)

2.5 Tabla partida_jugador (líneas 57–69)

Tabla intermedia de la relación muchos a muchos entre partida y jugador: una partida tiene 4 jugadores y un jugador participa en muchas partidas.

Fragmento de scriptuno.sql (líneas 57 a 69)

```
57  create table `unobd`.`partida_jugador` (
58      `id_partida` int not null,
59      `id_jugador` int not null,
60      `posicion` int not null,
61      primary key (`id_partida`, `id_jugador`),
62      unique key `uq_partida_posicion` (`id_partida`, `posicion`),
63      key `id_jugador` (`id_jugador`),
64      constraint `fk_pj_partida`
65          foreign key (`id_partida`)
66          references `partida` (`id_partida`),
67      constraint `fk_pj_jugador`
68          foreign key (`id_jugador`)
69          references `jugador` (`id_jugador`));
```

- Línea 61: llave primaria compuesta `(id_partida, id_jugador)`: un jugador no puede estar dos veces en la misma partida.
- Línea 60: y línea 62 `posicion` es el lugar en la mesa (1, 2, 3, 4) y define el orden de los turnos. `uq_partida_posicion` impide que dos jugadores tengan el mismo lugar en una partida.
- Líneas 64 a 69: dos llaves foráneas, una a `partida` y otra a `jugador`.
- Según el comentario de línea 55: sin esta tabla no se podrían contar las derrotas (solo se sabría quién ganó, no quién jugó).

2.6 Tabla jugada: el log (líneas 82–111)

Es la tabla más importante para el historial: cada movimiento de cada partida es una fila.

Fragmento de scriptuno.sql (líneas 71 a 111)

```
71  -- ============================================================
72  -- JUGADA: log de TODOS los movimientos de la partida.
73  -- carta_inicial -> carta volteada al inicio (sin jugador)
74  -- repartir -> carta que recibe el jugador en el reparto inicial
75  -- tirar -> el jugador tira id_carta
76  -- robar -> el jugador roba id_carta del mazo
77  -- pasar -> el jugador pasa turno (sin carta)
78  -- decir_uno -> el jugador canta UNO (sin carta)
79  -- penalizacion_uno -> no dijo UNO; roba id_carta como castigo
80  -- color_elegido solo aplica al tirar un cambiar_color o +4.
81  -- ============================================================
82  create table `unobd`.`jugada` (
83      `id_jugada` int not null auto_increment,
84      `id_partida` int not null,
85      `num_jugada` int not null,
86      `id_jugador` int null default null,
87      `tipo_accion` enum('carta_inicial', 'repartir', 'tirar', 'robar', 'pasar', 'decir_uno',
    'penalizacion_uno') not null,
88      `id_carta` int null default null,
89      `color_elegido` enum('rojo', 'amarillo', 'azul', 'verde') null default null,
90      `fecha_hora` timestamp not null default current_timestamp,
91      primary key (`id_jugada`),
92      unique key `uq_partida_num_jugada` (`id_partida`, `num_jugada`),
93      key `id_jugador` (`id_jugador`),
94      key `id_carta` (`id_carta`),
95      constraint `fk_jugada_partida`
96          foreign key (`id_partida`)
97          references `partida` (`id_partida`),
98      constraint `fk_jugada_pj`
99          foreign key (`id_partida`, `id_jugador`)
100         references `partida_jugador` (`id_partida`, `id_jugador`),
101     constraint `fk_jugada_carta`
102         foreign key (`id_carta`)
103         references `carta` (`id_carta`),
104     constraint chk_jugada_jugador check (
105         (tipo_accion = 'carta_inicial' and id_jugador is null)
106         or (tipo_accion <> 'carta_inicial' and id_jugador is not null)),
107     constraint chk_jugada_carta check (
108         (tipo_accion in ('carta_inicial', 'repartir', 'tirar', 'robar', 'penalizacion_uno')
    and id_carta is not null)
109         or (tipo_accion in ('pasar', 'decir_uno') and id_carta is null)),
110     constraint chk_jugada_color check (
111         color_elegido is null or tipo_accion in ('tirar', 'carta_inicial')));
```

- Línea 85: `num_jugada`: número consecutivo dentro de la partida (1, 2, 3...). Línea 92 `uq_partida_num_jugada` asegura que no se repita en una misma partida.
- Línea 86: `id_jugador` puede ser `null` solo para `carta_inicial` (la voltea "la mesa", no un jugador).
- Línea 87: `tipo_accion` y su significado están documentados en líneas 73 a 79:

tipo_accion	**¿Lleva carta?**	**Cuándo se registra**
`carta_inicial`	Sí (sin jugador)	Al crear la partida: la carta que se voltea.
`repartir`	Sí	Al crear la partida: 28 filas (7 cartas × 4 jugadores).

tipo_accion	¿Lleva carta?	Cuándo se registra
`tirar`	Sí	El jugador juega una carta. Si es comodín, también `color_elegido`.
`robar`	Sí	Clic en el mazo, o cartas recibidas por un +2/+4.
`pasar`	No	Botón "Pasar".
`decir_uno`	No	Presionó "¡UNO!" a tiempo.
`penalizacion_uno`	Sí	No dijo UNO: una fila por cada carta de castigo.

- Línea 89: `color_elegido`: solo se usa al tirar un comodín.
- Línea 90: `fecha_hora`: hora exacta del movimiento.
- Líneas 98 a 100: `fk_jugada_pj` es una llave foránea compuesta: el par `(id_partida, id_jugador)` debe existir en `partida_jugador`. Es decir, solo un jugador que participa en esa partida puede tener jugadas en ella. Si `id_jugador` es `null` (carta inicial), MySQL no revisa esta llave.
- Líneas 104 a 106: `chk_jugada_jugador`: carta_inicial sin jugador; todo lo demás con jugador.
- Líneas 107 a 109: `chk_jugada_carta`: pasar y decir_uno sin carta; todo lo demás con carta.
- Líneas 110 a 111: `chk_jugada_color`: solo `tirar` o `carta_inicial` pueden tener color elegido.

2.7 Vistas (líneas 118–143)

Una vista es una consulta guardada con nombre; se usa como si fuera una tabla (`select * from vista_log`).

Fragmento de scriptuno.sql (líneas 117 a 127)

```
117 -- Historial de ganadas / perdidas por jugador (solo partidas terminadas)
118 create view `unobd`.`vista_historial` as
119 select j.id_jugador,
120        j.nombre_jug,
121        count(p.id_partida) as partidas_jugadas,
122        coalesce(sum(p.id_ganador = j.id_jugador), 0) as ganadas,
123        coalesce(sum(p.id_ganador <> j.id_jugador), 0) as perdidas
124 from jugador j
125 left join partida_jugador pj on pj.id_jugador = j.id_jugador
126 left join partida p on p.id_partida = pj.id_partida and p.id_ganador is not null
127 group by j.id_jugador, j.nombre_jug;
```

vista_historial cuenta, por jugador, partidas jugadas, ganadas y perdidas.

- Líneas 124 a 126: `left join`: empieza en `jugador` para que aparezcan todos los jugadores, aunque no hayan jugado (con 0).

- Línea 126: la condición `p.id_ganador is not null` va en el `on` (no en un `where`): así solo se cuentan partidas con ganador, sin perder a los jugadores sin partidas. Las partidas abandonadas no cuentan.
- Línea 122: `sum(p.id_ganador = j.id_jugador)`: en MySQL una comparación vale 1 si es verdadera y 0 si es falsa, así que sumarla cuenta las victorias. `coalesce(..., 0)` cambia `null` por 0 cuando el jugador no tiene partidas.
- Línea 123: igual, pero cuenta las veces que ganó otro → derrotas.
- Línea 127: `group by`: una fila por jugador.

Fragmento de scriptuno.sql (líneas 129 a 143)

```
129 -- Log legible de movimientos
130 create view `unobd`.`vista_log` as
131 select ju.id_partida,
132        ju.num_jugada,
133        ju.fecha_hora,
134        coalesce(jg.nombre_jug, '(mesa)') as jugador,
135        ju.tipo_accion,
136        c.tipo_carta,
137        c.color_carta,
138        c.num_carta,
139        c.efecto,
140        ju.color_elegido
141 from jugada ju
142 left join jugador jg on jg.id_jugador = ju.id_jugador
143 left join carta c on c.id_carta = ju.id_carta;
```

vista_log convierte el log en algo legible: en lugar de ids muestra el nombre del jugador y los datos de la carta.

- Línea 134: `coalesce(jg.nombre_jug, '(mesa)')`: si no hay jugador (carta inicial) muestra "(mesa)".
- Líneas 142 a 143: `left join` para no perder las jugadas sin jugador o sin carta (pasar, decir_uno).

2.8 Datos: las 60 cartas (líneas 151–213)

Grupo	Líneas	Cantidad
Ordinarias 1–9 de los 4 colores (una de cada)	152–187	36
Comodín cambiar color	190–191	2
Comodín +4	192–193	2
Bloqueo, 2 por color	194–201	8
+2, 2 por color	202–209	8
Reversa, 1 por color	210–213	4
Total		60 (36 ordinarias + 24 especiales)

- Al insertar en ese orden, las ordinarias quedan con `id_carta` 1–36 y las especiales con 37–60.
- No hay cartas con número 0 (aunque el check lo permitiría). El UNO oficial tiene 108 cartas; el equipo decidió usar 60.
- Los jugadores no se insertan aquí (línea 149): se dan de alta desde el juego.

2.9 Diagrama entidad-relación

El diagrama se hizo en MySQL Workbench a partir de la base `unobd`. Cada cuadro es una tabla con sus columnas y cada línea es una llave foránea.

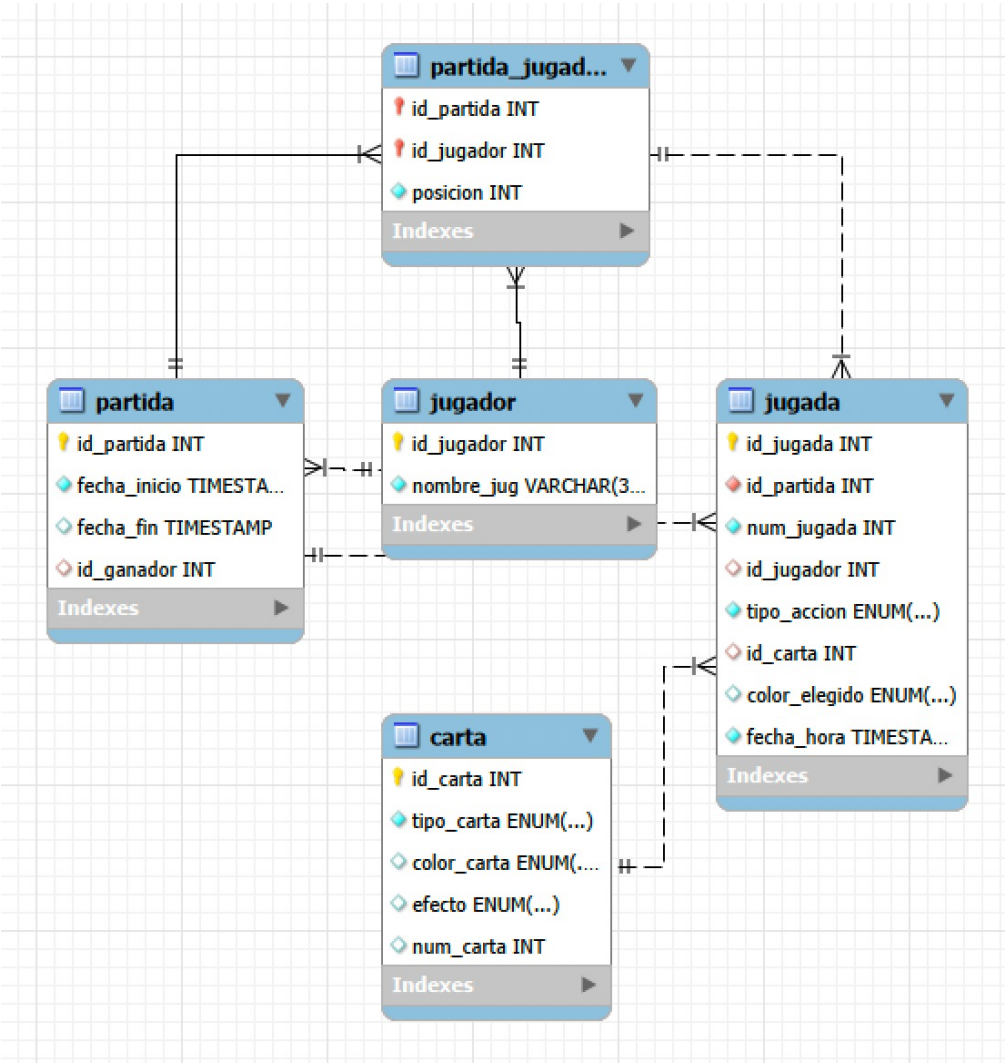


Figura 1. Diagrama entidad-relación de la base de datos unobd (MySQL Workbench).

Cómo se lee el diagrama:

- La llave amarilla marca la llave primaria de la tabla (por ejemplo `id_partida` en `partida`). La llave roja de `partida_jugador` indica que la llave primaria es compuesta y que además sus columnas son llaves foráneas (`id_partida` e `id_jugador`).
- El rombo marca una columna normal. Si está relleno, la columna es `not null`; si está vacío, puede ser `null` (por ejemplo `fecha_fin`, `id_ganador`, `color_carta`, `id_carta` en `jugada`). Si es rojizo, la columna es llave foránea.
- En los extremos de cada línea, las dos rayitas (||) significan "uno y solo uno" y la pata de gallo (el tridente) significa "muchos". Por ejemplo, entre `carta` y `jugada`: una carta puede aparecer en muchas jugadas, y cada jugada apunta a una sola carta.
- La línea continua es una relación identificadora: la llave foránea forma parte de la llave primaria de la tabla hija. Solo pasa con `partida_jugador`, cuya llave primaria es `(id_partida, id_jugador)`.
- La línea punteada es una relación no identificadora: la llave foránea es una columna normal (por ejemplo `id_ganador` en `partida` o `id_carta` en `jugada`).

Las mismas relaciones, en forma de tabla:

Relación	**Tipo**	**Cómo se implementa**
jugador — partida (participa)	Muchos a muchos	Tabla `partida_jugador` con llave compuesta
jugador — partida (gana)	Uno a muchos	`partida.id_ganador` → `jugador.id_jugador`
partida — jugada	Uno a muchos	`jugada.id_partida` → `partida.id_partida`
partida_jugador — jugada	Uno a muchos	FK compuesta `(id_partida, id_jugador)`
carta — jugada	Uno a muchos (opcional)	`jugada.id_carta` → `carta.id_carta` (puede ser null)

Nota: Si te preguntan "¿dónde se guarda la mano de cada jugador?": no se guarda directamente. Se puede reconstruir a partir del log: cartas recibidas (`repartir`, `robar`, `penalizacion_uno`) menos cartas tiradas (`tirar`).

3. API en Python (db.py y main.py)

3.1 db.py: la conexión a MySQL

Fragmento de db.py (líneas 1 a 20)

```
 1  import os
 2  import mysql.connector
 3  from mysql.connector import Error
 4  from dotenv import load_dotenv
 5
 6  load_dotenv()
 7  def get_connection():
 8      try:
 9          connection = mysql.connector.connect(
10              host=os.getenv("DB_HOST"),
11              user=os.getenv("DB_USER"),
12              password=os.getenv("DB_PASSWORD"),
13              database=os.getenv("DB_NAME"),
14              port=int(os.getenv("DB_PORT",3306)),
15              use_pure=True
16          )
17          if connection.is_connected():
18              return connection
19      except Error as e:
20          print(f"Error al conectar a MySQL: {e}")
```

- Líneas 1 a 4: imports: `os` para leer variables de entorno, `mysql.connector` para conectarse, `Error` para atrapar errores, `load_dotenv` para leer el archivo `.env`.
- Línea 6: `load_dotenv()` lee `.env` y pone sus valores como variables de entorno. Así la contraseña no está escrita en el código (antes sí estaba; ver el commit "se me olvido borrar mi clave" en el capítulo de commits).
- Línea 7: `get_connection()` abre y devuelve una conexión.
- Líneas 10 a 14: cada dato sale de `os.getenv`. Línea 14 el puerto se convierte a `int` y, si no está definido, usa 3306 (el puerto por defecto de MySQL).
- Línea 15: `use_pure=True` usa la implementación hecha 100 % en Python del conector (evita problemas con la versión compilada en C).
- Líneas 17 a 18: solo regresa la conexión si realmente está conectada.
- Líneas 19 a 21: si falla, imprime el error y devuelve `None`.

Observación: Si `connect` no lanza error pero `is_connected()` da `False`, la función termina sin `return` y Python devuelve `None` implícitamente. El resultado es el mismo que en el `except`, y `main.py` línea 31 lo maneja con un error 500.

3.2 main.py: imports y creación de la app (líneas 1–20)

Fragmento de main.py (líneas 1 a 20)

```
 1  from contextlib import contextmanager
 2  from typing import List, Literal, Optional
 3
 4  import mysql.connector
 5  from fastapi import FastAPI, HTTPException, Request
 6  from fastapi.responses import JSONResponse
 7  from pydantic import BaseModel, Field
 8
 9  from db import get_connection
10
11  app = FastAPI(
12      title="API del juego UNO",
13      description="Intermediario entre el juego en C# y la base de datos unobd en MySQL. "
14                  "Cada endpoint corresponde a una operacion de BaseDatos.cs.",
15  )
16
17  # Valores permitidos, iguales a los enum de la base de datos
18  Color = Literal["rojo", "amarillo", "azul", "verde"]
19  # 'carta_inicial' y 'repartir' solo se registran al crear la partida (POST /partidas)
20  Accion = Literal["tirar", "robar", "pasar", "decir_uno", "penalizacion_uno"]
```

- Línea 1: `contextmanager` permite crear funciones que se usan con `with` (ver `conexion()`).
- Línea 2: `List`, `Literal`, `Optional` son tipos para describir los datos esperados.
- Línea 5: de FastAPI: la clase de la app, `HTTPException` (para responder errores como 404) y `Request`.
- Línea 7: `BaseModel` y `Field` de Pydantic: describen la forma del JSON que llega y lo validan solos.
- Línea 9: importa la función de `db.py`.
- Líneas 11 a 15: crea la aplicación. El título y la descripción aparecen en `/docs`.
- Línea 18: `Color = Literal[...]`: solo esos 4 textos son válidos, igual que el `enum` de la base.
- Línea 20: `Accion`: las acciones que C# puede mandar. `carta_inicial` y `repartir` no están porque solo las registra la propia API al crear la partida.

3.3 Conexión y manejo de errores (líneas 27–55)

Fragmento de main.py (líneas 27 a 55)

```
27  @contextmanager
28  def conexion():
29      """Abre una conexion a MySQL y la cierra al terminar, aunque haya error."""
30      conn = get_connection()
31      if conn is None:
32          raise HTTPException(status_code=500, detail="Error de conexión a la base de datos")
33      try:
34          yield conn
35      finally:
```

```
36          if conn.is_connected():
37              conn.close()
38
39
40  @app.exception_handler(mysql.connector.Error)
41  def error_mysql(request: Request, e: mysql.connector.Error):
42      # Datos que violan llaves foraneas, unique o check (ej. un id que no existe) -> 400
43      # 3819 = se violo un 'check' de la tabla (ej. robar sin carta)
44      if isinstance(e, (mysql.connector.IntegrityError, mysql.connector.DataError)) or e.errno
    == 3819:
45          return JSONResponse(status_code=400, content={"detail": f"Datos inválidos:
    {e.msg}"})
46      return JSONResponse(status_code=500, content={"detail": f"Error de MySQL: {e.msg}"})
47
48
49  def partida_existente(cursor, id_partida: int) -> dict:
50      """Devuelve la partida o responde 404 si no existe."""
51      cursor.execute("select id_partida, fecha_fin from partida where id_partida = %s",
    (id_partida,))
52      partida = cursor.fetchone()
53      if partida is None:
54          raise HTTPException(status_code=404, detail=f"No existe la partida {id_partida}")
55      return partida
```

conexion() (líneas 27 a 37) es un context manager:

- Línea 30: abre la conexión. Líneas 31 a 32 si es `None`, responde error 500.
- Línea 34: `yield conn` entrega la conexión al bloque `with`. Cuando el bloque termina (bien o con error) se ejecuta el `finally` de líneas 35 a 37, que la cierra. Así nunca quedan conexiones abiertas.

error_mysql (líneas 40 a 46) es un manejador global: cualquier error de MySQL que no se atrape en un endpoint llega aquí.

- Línea 44: si es `IntegrityError` (llave foránea, unique), `DataError` (dato inválido) o el código 3819 (se violó un `check`), la culpa es de los datos enviados → 400 Bad Request.
- Línea 46: cualquier otro error de MySQL → 500 Internal Server Error.
- El mensaje va en el campo `"detail"`, que es el mismo formato que usa FastAPI; `BaseDatos.cs` lo lee en `LeerDetalle`.

partida_existente (líneas 49 a 55) busca una partida por id; si no existe responde 404. La usan varios endpoints para no repetir código. Devuelve también `fecha_fin` para saber si ya terminó.

`%s` en las consultas (línea 51 y todas las demás) son parámetros: el conector inserta el valor de forma segura. Esto evita la inyección SQL (nunca se pega texto del usuario directo en la consulta).

3.4 Modelos de entrada con Pydantic (líneas 62–88)

Fragmento de main.py (líneas 62 a 88)

```
62  class JugadorEntrada(BaseModel):
63      nombre: str = Field(min_length=1, max_length=30)
64
65
66  class CartaRepartida(BaseModel):
67      id_jugador: int
68      id_carta: int
69
70
71  class PartidaEntrada(BaseModel):
72      # Ids de los jugadores en el orden en que se sientan (posicion 1, 2, 3...)
73      jugadores: List[int] = Field(min_length=2)
74      # Cartas repartidas, en el orden en que se dieron
75      reparto: List[CartaRepartida]
76      id_carta_inicial: int
77
78
79  class JugadaEntrada(BaseModel):
80      id_jugador: int
81      tipo_accion: Accion
82      id_carta: Optional[int] = None
83      color_elegido: Optional[Color] = None
84
85
86  class TerminarEntrada(BaseModel):
87      # null = partida abandonada (se cerro el programa sin ganador)
88      id_ganador: Optional[int] = None
```

Cada clase describe el JSON que manda C#. Si el JSON no cumple, FastAPI responde automáticamente 422 Unprocessable Entity sin entrar a la función.

Modelo	**Usado en**	**Reglas**
`JugadorEntrada`	POST /jugadores	`nombre` de 1 a 30 caracteres (línea 63)
`CartaRepartida`	dentro de `PartidaEntrada`	un par jugador–carta
`PartidaEntrada`	POST /partidas	al menos 2 jugadores (línea 73), la lista del reparto y la carta inicial
`JugadaEntrada`	POST /partidas/{id}/jugadas	`tipo_accion` debe ser una `Accion`; carta y color opcionales; el color debe ser un `Color`
`TerminarEntrada`	PUT /partidas/{id}/terminar	`id_ganador` opcional (null = abandonada)

3.5 Endpoints

Método y ruta	Función	Líneas	Lo usa C# en
GET `/`	`estado`	95–97	`ApiDisponible` (FormInicio)
GET `/cartas`	`obtener_cartas`	100–106	`ObtenerCartas`
POST `/jugadores`	`obtener_o_crear_jugador`	109–124	`ObtenerOCrearJugador`
POST `/partidas`	`registrar_partida`	127–158	`RegistrarPartida`
GET `/partidas/{id}/posiciones`	`obtener_posiciones`	161–169	`ObtenerPosiciones`
POST `/partidas/{id}/jugadas`	`registrar_jugada`	172–189	`RegistrarJugada`
PUT `/partidas/{id}/terminar`	`terminar_partida`	192–205	`TerminarPartida`
GET `/partidas/{id}/log`	`obtener_log`	208–214	(solo consulta, no lo usa el juego)
GET `/historial`	`obtener_historial`	217–222	(solo consulta)
GET `/ganadores`	`obtener_ganadores`	225–233	(solo consulta)

GET / y GET /cartas

Fragmento de main.py (líneas 95 a 106)

```
 95 @app.get("/", summary="Comprobar que la API esta corriendo")
 96 def estado():
 97     return {"estado": "ok"}
 98
 99
100 @app.get("/cartas", summary="Todas las cartas del mazo")
101 def obtener_cartas():
102     with conexion() as conn:
103         cursor = conn.cursor(dictionary=True)
104         cursor.execute(
105             "select id_carta, tipo_carta, color_carta, efecto, num_carta from carta order by
    id_carta")
106         return cursor.fetchall()
```

- Línea 95: el decorador `@app.get("/")` registra la función para esa ruta. Responde `{"estado": "ok"}`; sirve solo para saber si la API está viva.
- Línea 103: `cursor(dictionary=True)` hace que cada fila sea un diccionario `{"id_carta": 1, ...}` en lugar de una tupla; FastAPI lo convierte a JSON con esos nombres.

- Líneas 105 a 106: devuelve las 60 cartas ordenadas por id.

POST /jugadores: buscar o crear

Fragmento de main.py (líneas 109 a 124)

```
@app.post("/jugadores", summary="Buscar un jugador por nombre, o darlo de alta si no existe")
def obtener_o_crear_jugador(datos: JugadorEntrada):
    nombre = datos.nombre.strip()
    if not nombre:
        raise HTTPException(status_code=400, detail="El nombre no puede estar vacío")

    with conexion() as conn:
        cursor = conn.cursor(dictionary=True)
        cursor.execute("select id_jugador, nombre_jug from jugador where nombre_jug = %s", (nombre,))
        jugador = cursor.fetchone()
        if jugador is not None:
            return jugador

        cursor.execute("insert into jugador (nombre_jug) values (%s)", (nombre,))
        conn.commit()
        return {"id_jugador": cursor.lastrowid, "nombre_jug": nombre}
```

- Líneas 111 a 113: quita espacios; si queda vacío → 400.
- Líneas 117 a 120: si ya existe un jugador con ese nombre, lo devuelve (así se reutiliza su historial).
- Líneas 122 a 124: si no existe, lo inserta, hace `commit` (guarda definitivamente) y devuelve el id nuevo con `lastrowid`.

POST /partidas: crear la partida en una transacción

Fragmento de main.py (líneas 127 a 158)

```
@app.post("/partidas", summary="Crear una partida con sus jugadores, el reparto y la carta inicial")
def registrar_partida(datos: PartidaEntrada):
    if len(set(datos.jugadores)) != len(datos.jugadores):
        raise HTTPException(status_code=400, detail="Hay jugadores repetidos")
    if any(c.id_jugador not in datos.jugadores for c in datos.reparto):
        raise HTTPException(status_code=400, detail="El reparto incluye jugadores que no están en la partida")

    with conexion() as conn:
        cursor = conn.cursor()
        try:
            # Todo en una sola transaccion: si algo falla no queda una partida a medias
            cursor.execute("insert into partida (fecha_inicio) values (now())")
            id_partida = cursor.lastrowid

            cursor.executemany(
                "insert into partida_jugador (id_partida, id_jugador, posicion) values (%s, %s, %s)",
                [(id_partida, id_jugador, posicion)
                 for posicion, id_jugador in enumerate(datos.jugadores, start=1)])

            jugadas = [(id_partida, num, c.id_jugador, "repartir", c.id_carta)
                       for num, c in enumerate(datos.reparto, start=1)]
```

```
148             jugadas.append((id_partida, len(datos.reparto) + 1, None, "carta_inicial",
    datos.id_carta_inicial))
149             cursor.executemany(
150                 "insert into jugada (id_partida, num_jugada, id_jugador, tipo_accion,
    id_carta) "
151                 "values (%s, %s, %s, %s, %s)", jugadas)
152 
153             conn.commit()
154         except Exception:
155             conn.rollback()
156             raise
157 
158         return {"id_partida": id_partida}
```

- Líneas 129 a 130: `set` quita repetidos: si el tamaño cambia, había jugadores repetidos → 400.
- Líneas 131 a 132: cada carta repartida debe ser de un jugador de la partida.
- Líneas 136 a 156: todo dentro de un `try`: es una transacción. Si cualquier `insert` falla, línea 155 `rollback()` deshace todo y no queda una partida a medias. Si todo sale bien, línea 153 `commit()` guarda todo junto.
- Líneas 138 a 139: crea la partida y obtiene su id.
- Líneas 141 a 144: `executemany` inserta varias filas de una vez. `enumerate(datos.jugadores, start=1)` asigna la posición 1, 2, 3, 4 según el orden en que llegaron los ids.
- Líneas 146 a 147: arma una jugada `repartir` por cada carta del reparto, numeradas desde 1 (28 en total).
- Línea 148: agrega al final la `carta_inicial`, sin jugador (`None`) y con número 29.
- Línea 156: `raise` vuelve a lanzar el error para que lo atrape `error_mysql` y responda 400/500.

GET /partidas/{id}/posiciones

Fragmento de main.py (líneas 161 a 169)

```
161 @app.get("/partidas/{id_partida}/posiciones", summary="Lugar en la mesa de cada jugador de
    la partida")
162 def obtener_posiciones(id_partida: int):
163     with conexion() as conn:
164         cursor = conn.cursor(dictionary=True)
165         partida_existente(cursor, id_partida)
166         cursor.execute(
167             "select id_jugador, posicion from partida_jugador where id_partida = %s order by
    posicion",
168             (id_partida,))
169         return cursor.fetchall()
```

`{id_partida}` en la ruta se convierte en el parámetro `id_partida: int`. Primero verifica que la partida exista (404 si no) y devuelve la lista `[{id_jugador, posicion}]` ordenada.

POST /partidas/{id}/jugadas: registrar un movimiento

Fragmento de main.py (líneas 172 a 189)

```
172 @app.post("/partidas/{id_partida}/jugadas", summary="Registrar un movimiento en el log")
173 def registrar_jugada(id_partida: int, datos: JugadaEntrada):
174     with conexion() as conn:
175         cursor = conn.cursor(dictionary=True)
176         partida = partida_existente(cursor, id_partida)
177         if partida["fecha_fin"] is not None:
178             raise HTTPException(status_code=409, detail=f"La partida {id_partida} ya
    terminó")
179 
180         # num_jugada = siguiente numero dentro de la partida
181         cursor.execute(
182             "insert into jugada (id_partida, num_jugada, id_jugador, tipo_accion, id_carta,
    color_elegido) "
183             "select %s, coalesce(max(num_jugada), 0) + 1, %s, %s, %s, %s from jugada where
    id_partida = %s",
184             (id_partida, datos.id_jugador, datos.tipo_accion, datos.id_carta,
    datos.color_elegido, id_partida))
185         conn.commit()
186         id_jugada = cursor.lastrowid
187 
188         cursor.execute("select num_jugada from jugada where id_jugada = %s", (id_jugada,))
189         return {"id_jugada": id_jugada, "num_jugada": cursor.fetchone()["num_jugada"]}
```

- Líneas 177 a 178: si la partida ya tiene `fecha_fin`, responde 409 Conflict: no se pueden agregar jugadas a una partida terminada.
- Líneas 181 a 184: el `insert ... select` calcula el siguiente número de jugada dentro de la misma consulta: `coalesce(max(num_jugada), 0) + 1`. Si no hubiera jugadas, `max` da `null` y `coalesce` lo cambia por 0.
- Líneas 188 a 189: vuelve a leer `num_jugada` para devolverlo.

Nota: Las reglas de la jugada (que la carta tenga carta si es `tirar`, que el jugador pertenezca a la partida, etc.) las hacen cumplir los `check` y llaves foráneas de MySQL. La API no repite esas validaciones; si fallan, `error_mysql` responde 400.

PUT /partidas/{id}/terminar

Fragmento de main.py (líneas 192 a 205)

```
192 @app.put("/partidas/{id_partida}/terminar", summary="Terminar la partida, con o sin
    ganador")
193 def terminar_partida(id_partida: int, datos: TerminarEntrada):
194     with conexion() as conn:
195         cursor = conn.cursor(dictionary=True)
196         partida = partida_existente(cursor, id_partida)
197         # Si ya estaba terminada no se cambia nada (por ejemplo, al cerrar el programa
    despues de ganar)
198         if partida["fecha_fin"] is not None:
199             return {"id_partida": id_partida, "terminada_ahora": False}
200 
201         cursor.execute(
202             "update partida set fecha_fin = now(), id_ganador = %s where id_partida = %s",
203             (datos.id_ganador, id_partida))
204         conn.commit()
205         return {"id_partida": id_partida, "terminada_ahora": True}
```

- Líneas 198 a 199: si ya estaba terminada no cambia nada y responde `terminada_ahora: False`. Esto lo hace idempotente: llamarlo dos veces no causa error. Pasa, por ejemplo, cuando alguien gana (se termina con ganador) y después se cierra la ventana (se intenta terminar sin ganador).
- Líneas 201 a 203: pone la fecha de fin y el ganador (o `null`).

Consultas: log, historial y ganadores

Fragmento de main.py (líneas 208 a 233)

```
208 @app.get("/partidas/{id_partida}/log", summary="Movimientos de una partida (vista_log)")
209 def obtener_log(id_partida: int):
210     with conexion() as conn:
211         cursor = conn.cursor(dictionary=True)
212         partida_existente(cursor, id_partida)
213         cursor.execute("select * from vista_log where id_partida = %s order by num_jugada",
    (id_partida,))
214         return cursor.fetchall()
215
216
217 @app.get("/historial", summary="Partidas jugadas, ganadas y perdidas por jugador
    (vista_historial)")
218 def obtener_historial():
219     with conexion() as conn:
220         cursor = conn.cursor(dictionary=True)
221         cursor.execute("select * from vista_historial order by ganadas desc, nombre_jug")
222         return cursor.fetchall()
223
224
225 @app.get("/ganadores", summary="Partidas terminadas con su ganador")
226 def obtener_ganadores():
227     with conexion() as conn:
228         cursor = conn.cursor(dictionary=True)
229         cursor.execute(
230             "select p.id_partida, p.fecha_inicio, p.fecha_fin, p.id_ganador, j.nombre_jug as
    ganador "
231             "from partida p join jugador j on j.id_jugador = p.id_ganador "
232             "order by p.fecha_fin desc")
233         return cursor.fetchall()
```

- `/partidas/{id}/log` usa la vista `vista_log`.
- `/historial` usa `vista_historial`, ordenado por más victorias.
- `/ganadores` une `partida` con `jugador` (línea 231) para mostrar el nombre del ganador. Es un `join` normal, por lo que las partidas sin ganador no aparecen.

3.6 Códigos HTTP que puede devolver la API

Código	Significado	Cuándo
200	OK	Todo salió bien
400	Bad Request	Datos inválidos: nombre vacío, jugadores repetidos, violación de FK/unique/check

Código	Significado	Cuándo
404	Not Found	La partida no existe
409	Conflict	Se intenta registrar una jugada en una partida terminada
422	Unprocessable Entity	El JSON no cumple el modelo Pydantic (ej. color "morado")
500	Internal Server Error	No hay conexión con MySQL u otro error del servidor

4. C#: punto de entrada, modelos y comunicación con la API

4.1 Conceptos de C# que aparecen en todo el proyecto

Concepto	Qué significa
`namespace Juego_UNO`	Agrupa todas las clases del proyecto bajo un mismo nombre.
`public partial class Ventana : Form`	`partial`: la clase está repartida en dos archivos (`Form1.cs` y `Form1.Designer.cs`). `: Form` significa que hereda de la ventana de Windows Forms.
`{ get; set; }`	Propiedad automática: un campo que se puede leer y escribir.
`int?`	Entero que también puede ser `null` (se llama nullable).
`List<T>`, `Dictionary<K,V>`	Lista dinámica y diccionario (clave → valor).
`const` / `static readonly`	Valor fijo. `const` se decide al compilar; `static readonly` al iniciar el programa (sirve para objetos como `Size`).
`(s, e) => ...`	Expresión lambda: una función pequeña sin nombre. Se usa para manejar eventos.
`+=` en un evento	Suscribe una función al evento (ej. `Click += ...`).
`using (var x = ...) { }`	Libera el recurso (llama `Dispose`) al salir del bloque.
LINQ: `Select`, `Where`, `OrderBy`, `ToList`, `Sum`	Operaciones sobre colecciones (transformar, filtrar, ordenar).
`$"Hola {nombre}"`	Cadena interpolada: mete variables dentro del texto.
`??`	Si lo de la izquierda es `null`, usa lo de la derecha.
`try / catch`	Atrapa errores para que el programa no se cierre.
`async` / `await`	Esperar algo (ej. un `Task.Delay`) sin congelar la ventana.

4.2 Program.cs: el punto de entrada

Fragmento de Program.cs (líneas 9 a 21)

```
 9  internal static class Program
10  {
11      /// <summary>
12      /// Punto de entrada principal para la aplicación.
13      /// </summary>
14      [STAThread]
15      static void Main()
16      {
17          Application.EnableVisualStyles();
18          Application.SetCompatibleTextRenderingDefault(false);
19          Application.Run(new FormInicio());
20      }
21  }
```

- Línea 14: `[STAThread]`: Windows Forms necesita que el hilo principal sea de tipo STA (requisito de los controles de Windows).
- Línea 15: `Main` es lo primero que se ejecuta.
- Línea 17: activa el estilo visual moderno de Windows en los controles.
- Línea 18: usa el motor de texto moderno (GDI) en lugar del antiguo.
- Línea 19: `Application.Run(new FormInicio())` abre la pantalla de inicio y mantiene el programa vivo hasta que se cierre esa ventana.

4.3 Carta.cs: el modelo de una carta

Fragmento de Carta.cs (líneas 6 a 36)

```
 6  public class Carta
 7  {
 8      public int Id { get; set; }
 9      public string Tipo { get; set; } // "ordinaria" o "especial"
10      public string Color { get; set; } // null en los comodines
11      public string Efecto { get; set; } // null en las ordinarias
12      public int? Numero { get; set; } // null en las especiales
13
14      public override string ToString()
15      {
16          if (Tipo == "ordinaria")
17              return Capitalizar(Color) + " " + Numero;
18
19          switch (Efecto)
20          {
21              case "cambiar_color": return "Comodín cambio de color";
22              case "+4": return "Comodín +4";
23              case "bloquear": return Capitalizar(Color) + " bloqueo";
24              case "cambiar_direccion": return Capitalizar(Color) + " reversa";
25              case "+2": return Capitalizar(Color) + " +2";
26              default: return Efecto;
27          }
28      }
29
30      private static string Capitalizar(string texto)
31      {
32          if (string.IsNullOrEmpty(texto))
33              return texto;
34          return char.ToUpper(texto[0]) + texto.Substring(1);
35      }
36  }
```

- Líneas 8 a 12: las 5 propiedades corresponden exactamente a las 5 columnas de la tabla `carta`. `Numero` es `int?` porque las especiales no tienen número.
- Líneas 14 a 28: `ToString()` está sobrescrito (`override`) para que una carta se pueda escribir como texto legible. Se usa en el mensaje de jugada inválida: "No puedes jugar Rojo 5 sobre Azul 3".
- Líneas 16 a 17: ordinaria → "Rojo 5". Líneas 19 a 27 `switch` según el efecto.
- Líneas 30 a 35: `Capitalizar` convierte la primera letra en mayúscula ("rojo" → "Rojo").

4.4 Jugador.cs: el modelo de un jugador

Fragmento de Jugador.cs (líneas 8 a 19)

```
 8  public class Jugador
 9  {
10      public int Id { get; set; }
11      public string Nombre { get; set; }
12      public int Posicion { get; set; } // lugar en la mesa (partida_jugador.posicion),
    empieza en 1
13      public List<Carta> Mano { get; } = new List<Carta>();
14
15      public override string ToString()
16      {
17          return Nombre;
18      }
19  }
```

- Líneas 10 a 11: id y nombre, igual que la tabla `jugador`.
- Línea 12: `Posicion`: su lugar en la mesa (1 a 4), viene de `partida_jugador`.
- Línea 13: `Mano` es la lista de cartas que tiene. Solo tiene `get` (no `set`): la lista se crea una vez y después solo se le agregan o quitan cartas. No existe en la base de datos.
- Líneas 15 a 18: `ToString` devuelve el nombre.

4.5 BaseDatos.cs: el cliente de la API

Esta clase es el "puente" entre el juego y la API. Es `static`: no se crean objetos, se llama directamente `BaseDatos.Metodo(...)`.

La excepción ErrorBaseDatos (líneas 16–19)

Fragmento de BaseDatos.cs (líneas 12 a 19)

```
12  /// <summary>
13  /// Error al comunicarse con la API: no esta corriendo, o respondio con un error.
14  /// Reemplaza a MySqlException en los catch de Form1.cs.
15  /// </summary>
16  public class ErrorBaseDatos : Exception
17  {
18      public ErrorBaseDatos(string mensaje, Exception causa = null) : base(mensaje, causa) { }
19  }
```

Una excepción propia que hereda de `Exception`. Cualquier problema con la API (apagada, lenta, o respondió con error) se convierte en `ErrorBaseDatos`. Así `Form1.cs` solo necesita un tipo de `catch`. El comentario (línea 14) recuerda que antes se usaba `MySqlException`, cuando C# se conectaba directo a MySQL.

Configuración del cliente HTTP (líneas 25–35)

Fragmento de BaseDatos.cs (líneas 25 a 35)

```
25  public static class BaseDatos
26  {
27      // Direccion donde corre la API (uvicorn main:app --reload)
28      private const string UrlApi = "http://127.0.0.1:8000/";
29
30      // Un solo HttpClient para todo el programa (crear uno por peticion es lento)
31      private static readonly HttpClient cliente = new HttpClient
32      {
33          BaseAddress = new Uri(UrlApi),
34          Timeout = TimeSpan.FromSeconds(10)
35      };
```

- Línea 28: la dirección de la API está fija en el código: `http://127.0.0.1:8000/` (la misma computadora, puerto 8000 de uvicorn).
- Líneas 31 a 35: un solo `HttpClient` para todo el programa (crear uno por petición es lento y gasta conexiones). `BaseAddress` permite usar rutas cortas como `"cartas"`. `Timeout` de 10 s: si la API no responde en ese tiempo se cancela.

Métodos públicos (líneas 44–147)

Fragmento de BaseDatos.cs (líneas 44 a 65)

```
44  public static bool ApiDisponible()
45  {
46      try
47      {
48          Get<JObject>("");
49          return true;
50      }
51      catch (ErrorBaseDatos)
52      {
53          return false;
54      }
55  }
56
57  /// <summary>
58  /// Busca al jugador por nombre; si no existe lo da de alta.
59  /// POST /jugadores
60  /// </summary>
61  public static Jugador ObtenerOCrearJugador(string nombre)
62  {
63      var respuesta = Post<JugadorApi>("jugadores", new { nombre });
64      return new Jugador { Id = respuesta.IdJugador, Nombre = respuesta.NombreJug };
65  }
```

- ApiDisponible (líneas 44 a 55): hace `GET /`. Si responde, `true`; si lanza `ErrorBaseDatos`, `false`. Lo usa FormInicio para mostrar "Conectado".
- ObtenerOCrearJugador (líneas 61 a 65): `new { nombre }` es un objeto anónimo que se convierte en el JSON `{"nombre": "Ana"}`. La respuesta se lee como `JugadorApi` y se convierte a `Jugador`.

Fragmento de BaseDatos.cs (líneas 71 a 83)

```
71  public static List<Carta> ObtenerCartas()
72  {
73      return Get<List<CartaApi>>("cartas")
74          .Select(c => new Carta
75          {
76              Id = c.IdCarta,
77              Tipo = c.TipoCarta,
78              Color = c.ColorCarta,
79              Efecto = c.Efecto,
80              Numero = c.NumCarta
81          })
82          .ToList();
83  }
```

- ObtenerCartas: `GET /cartas`, recibe una lista de `CartaApi` y con LINQ `Select` convierte cada una a `Carta`.

Fragmento de BaseDatos.cs (líneas 91 a 108)

```
91  public static int RegistrarPartida(List<Jugador> jugadores, Carta cartaInicial)
92  {
93      // Se manda en el mismo orden en que se reparte: una carta por jugador por vuelta
94      var reparto = new List<object>();
95      for (int vuelta = 0; vuelta < jugadores[0].Mano.Count; vuelta++)
96      {
97          foreach (var jugador in jugadores)
98              reparto.Add(new { id_jugador = jugador.Id, id_carta = jugador.Mano[vuelta].Id
    });
99      }
100
101     var datos = new
102     {
103         jugadores = jugadores.Select(j => j.Id).ToList(), // en orden de posicion
104         reparto,
105         id_carta_inicial = cartaInicial.Id
106     };
107     return Post<PartidaApi>("partidas", datos).IdPartida;
108 }
```

- RegistrarPartida: arma el reparto en el mismo orden en que se repartió: vuelta por vuelta (línea 95), una carta por jugador (líneas 97 a 98). `jugador.Mano[vuelta]` es la carta que recibió en esa vuelta. Así el log queda en el orden real.
- Líneas 101 a 106: el JSON lleva los ids de los jugadores en orden de posición, el reparto y el id de la carta inicial. Nota que los nombres son snake_case (`id_carta_inicial`) para que coincidan con el modelo Pydantic.
- Línea 107: devuelve solo el `IdPartida`.

Fragmento de BaseDatos.cs (líneas 114 a 147)

```
114 public static Dictionary<int, int> ObtenerPosiciones(int idPartida)
115 {
116     return Get<List<PosicionApi>>($"partidas/{idPartida}/posiciones")
117         .ToDictionary(p => p.IdJugador, p => p.Posicion);
118 }
119
120 /// <summary>
121 /// Guarda un movimiento en el log (tabla jugada). La API calcula el numero de jugada.
```

```
122 /// tipoAccion: "tirar", "robar", "pasar", "decir_uno" o "penalizacion_uno".
123 /// colorElegido solo se usa al tirar un comodin (cambiar_color o +4).
124 /// POST /partidas/{id}/jugadas
125 /// </summary>
126 public static void RegistrarJugada(int idPartida, int idJugador, string tipoAccion, int?
    idCarta,
127     string colorElegido = null)
128 {
129     var datos = new
130     {
131         id_jugador = idJugador,
132         tipo_accion = tipoAccion,
133         id_carta = idCarta,
134         color_elegido = colorElegido
135     };
136     Post<JObject>($"partidas/{idPartida}/jugadas", datos);
137 }
138
139 /// <summary>
140 /// Marca la partida como terminada. Si idGanador es null, la partida queda
141 /// abandonada (sin ganador). No hace nada si la partida ya estaba terminada.
142 /// PUT /partidas/{id}/terminar
143 /// </summary>
144 public static void TerminarPartida(int idPartida, int? idGanador)
145 {
146     Put<JObject>($"partidas/{idPartida}/terminar", new { id_ganador = idGanador });
147 }
```

- ObtenerPosiciones (líneas 114 a 118): convierte la lista en un diccionario `id_jugador → posicion` con `ToDictionary`.
- RegistrarJugada (líneas 126 a 137): `int? idCarta` porque pasar y decir UNO no llevan carta. `colorElegido = null` es un parámetro opcional. El resultado se ignora (`JObject` genérico).
- TerminarPartida (líneas 144 a 147): `PUT` con `id_ganador` o `null`.

El corazón: Enviar<T> (líneas 153–198)

Fragmento de BaseDatos.cs (líneas 153 a 198)

```
153 private static T Get<T>(string ruta) => Enviar<T>(HttpMethod.Get, ruta, null);
154 private static T Post<T>(string ruta, object datos) => Enviar<T>(HttpMethod.Post, ruta,
    datos);
155 private static T Put<T>(string ruta, object datos) => Enviar<T>(HttpMethod.Put, ruta,
    datos);
156
157 /// <summary>
158 /// Manda la peticion a la API y convierte la respuesta JSON al tipo T.
159 /// Cualquier falla (API apagada, error 400/404/409/500) se lanza como ErrorBaseDatos.
160 /// </summary>
161 private static T Enviar<T>(HttpMethod metodo, string ruta, object datos)
162 {
163     using (var peticion = new HttpRequestMessage(metodo, ruta))
164     {
165         if (datos != null)
166         {
167             string json = JsonConvert.SerializeObject(datos);
168             peticion.Content = new StringContent(json, Encoding.UTF8, "application/json");
169         }
170
```

```
171         HttpResponseMessage respuesta;
172         string cuerpo;
173         try
174         {
175             // Task.Run evita que la ventana se bloquee esperando la respuesta (deadlock)
176             respuesta = Task.Run(() =>
    cliente.SendAsync(peticion)).GetAwaiter().GetResult();
177             cuerpo = Task.Run(() =>
    respuesta.Content.ReadAsStringAsync()).GetAwaiter().GetResult();
178         }
179         catch (HttpRequestException ex)
180         {
181             throw new ErrorBaseDatos(
182                 $"No se pudo conectar con la API en {UrlApi}\n¿Está corriendo uvicorn?",
    ex);
183         }
184         catch (TaskCanceledException ex)
185         {
186             throw new ErrorBaseDatos("La API tardó demasiado en responder.", ex);
187         }
188 
189         using (respuesta)
190         {
191             if (!respuesta.IsSuccessStatusCode)
192                 throw new ErrorBaseDatos(
193                     $"La API respondió con error
    {(int)respuesta.StatusCode}:\n{LeerDetalle(cuerpo)}");
194 
195             return JsonConvert.DeserializeObject<T>(cuerpo);
196         }
197     }
198 }
```

- Líneas 153 a 155: `Get`, `Post` y `Put` son atajos (sintaxis `=>`) que llaman a `Enviar` con el método HTTP correspondiente.
- Línea 161: `<T>` es un genérico: el método sirve para cualquier tipo de respuesta (`List<CartaApi>`, `JugadorApi`...).
- Línea 163: crea la petición dentro de `using` para liberarla al final.
- Líneas 165 a 169: si hay datos, `JsonConvert.SerializeObject` (librería Newtonsoft.Json) los convierte a texto JSON y se pone como cuerpo con tipo `application/json`.
- Líneas 176 a 177: manda la petición y lee la respuesta. `SendAsync` es asíncrono; `.GetAwaiter().GetResult()` espera el resultado. Se envuelve en `Task.Run` para que corra en otro hilo: si se esperara directamente desde el hilo de la ventana se produciría un deadlock (la ventana se congela para siempre).
- Líneas 179 a 183: `HttpRequestException` = no se pudo conectar (uvicorn apagado).
- Líneas 184 a 187: `TaskCanceledException` = se cumplió el timeout de 10 s.
- Líneas 191 a 193: si el código no es 2xx (por ejemplo 400 o 404), lanza `ErrorBaseDatos` con el número y el mensaje de la API.
- Línea 195: si todo salió bien, `DeserializeObject<T>` convierte el JSON al tipo pedido.

LeerDetalle (líneas 203–220)

Fragmento de BaseDatos.cs (líneas 203 a 220)

```
203 private static string LeerDetalle(string cuerpo)
204 {
205     try
206     {
207         JToken detalle = JObject.Parse(cuerpo)["detail"];
208         if (detalle == null)
209             return cuerpo;
210         if (detalle.Type == JTokenType.String)
211             return (string)detalle;
212 
213         // Error 422 (datos con formato invalido): FastAPI manda una lista de errores
214         return string.Join("\n", detalle.Select(d => (string)d["msg"]));
215     }
216     catch (JsonException)
217     {
218         return cuerpo;
219     }
220 }
```

Saca el mensaje de error del campo `"detail"`. Si es texto (errores 400/404/409/500) lo devuelve tal cual (líneas 210 a 211). Si es una lista (error 422 de Pydantic), junta todos los `"msg"` en líneas separadas (línea 214). Si la respuesta ni siquiera es JSON, devuelve el texto crudo (líneas 216 a 219).

Clases DTO (líneas 226–250)

Fragmento de BaseDatos.cs (líneas 226 a 250)

```
226 private class CartaApi
227 {
228     [JsonProperty("id_carta")] public int IdCarta { get; set; }
229     [JsonProperty("tipo_carta")] public string TipoCarta { get; set; }
230     [JsonProperty("color_carta")] public string ColorCarta { get; set; }
231     [JsonProperty("efecto")] public string Efecto { get; set; }
232     [JsonProperty("num_carta")] public int? NumCarta { get; set; }
233 }
234 
235 private class JugadorApi
236 {
237     [JsonProperty("id_jugador")] public int IdJugador { get; set; }
238     [JsonProperty("nombre_jug")] public string NombreJug { get; set; }
239 }
240 
241 private class PartidaApi
242 {
243     [JsonProperty("id_partida")] public int IdPartida { get; set; }
244 }
245 
246 private class PosicionApi
247 {
248     [JsonProperty("id_jugador")] public int IdJugador { get; set; }
249     [JsonProperty("posicion")] public int Posicion { get; set; }
250 }
```

Son clases privadas que solo describen la forma del JSON. `[JsonProperty("id_carta")]` le dice a Newtonsoft que la propiedad `IdCarta` corresponde a la clave `"id_carta"` del JSON (en C# se usa PascalCase y en Python/SQL snake_case). Después se convierten a `Carta`/`Jugador`, así el resto del juego no depende del formato de la API.

5. Form1.cs: la clase Ventana (lógica del juego)

Este es el archivo más importante (1021 líneas). Contiene todas las reglas, el estado de la partida y el acomodo visual de la mesa. Está organizado así:

Sección	Líneas	Contenido
Constantes y estado	17–70	Tamaños, mazo, manos, turno, dirección, cuenta de UNO, controles, caché de imágenes
Constructor y estilo	72–179	`Ventana()`, `AplicarEstilo`, `CrearIndicadorTurno`
Inicio de partida	184–242	`EmpezarPartida`
Jugadas	244–413	`JugarCarta`, `EfectoCartaInvalida`, `botonPasar_Click`, `PedirColor`, `AplicarEfecto`
Turnos y cartas	418–563	`Siguiente`, `DarCartas`, `TerminarConGanador`, `AnunciarGanador`, `Ventana_FormClosing`, `Barajar`, `Repartir`, `SacarCartaInicial`, `RegresarAlMazo`
Dibujo	565–703	`MostrarManos`, `MostrarMano`, `ActualizarEtiqueta`, `MostrarDescarte`, `ActualizarTurno`, `ActualizarTitulo`, `MostrarErrorBaseDatos`
Imágenes	709–768	`NombreArchivo`, `ObtenerImagenCarta`, `CargarImagen`, `MostrarCartaEnPictureBox`
Acomodo de la mesa	777–913	`EsLateral`, `GiroCartas`, `MargenCarta`, `AcomodarLayout`, `AjustarMano`, `CentrarDebajo`, `Capitalizar`, `ActivarDobleBuffer`
Robar	915–933	`RobarCarta`
Decir UNO	943–1019	`IniciarCuentaUno`, `AvanzarCuentaUno`, `DecirUno`, `DetenerCuentaUno`, `ActualizarTextoUno`, `AcomodarBotonUno`

5.1 Constantes y variables de estado (líneas 15–70)

Fragmento de Form1.cs (líneas 15 a 70)

```
15  public partial class Ventana : Form
16  {
17      private const int NumJugadores = 4;
18      private const int CartasPorJugador = 7;
19
20      // Tamaño de las cartas en la mano (en los lados se muestran giradas)
21      private const int AnchoCarta = 80;
22      private const int AltoCarta = 120;
23      // Cuanto "sube" una carta al pasar el mouse sobre ella
24      private const int Elevacion = 12;
25      // Tamaño de las cartas del centro (mazo y descarte)
26      private static readonly Size TamañoCentro = new Size(100, 150);
27
28      private static readonly Random aleatorio = new Random();
```

```
29
30      // Ordenados por su posicion en la mesa (partida_jugador.posicion)
31      private List<Jugador> jugadores = new List<Jugador>();
32      private List<Carta> mazo = new List<Carta>();
33      // Carta boca arriba (pictureBox4). Al jugarse otra encima, esta regresa al mazo.
34      private Carta cartaArriba;
35      // Color que vale ahora: el de cartaArriba, o el elegido si es un comodin.
36      // Se guarda aparte para no modificar la carta (los comodines no tienen color propio).
37      private string colorActual;
38      private int idPartida;
39      private bool partidaEnCurso;
40
41      // Indice (en jugadores) de quien tiene el turno, y sentido del juego:
42      // 1 = posiciones ascendentes (1, 2, 3, 4), -1 = descendentes (4, 3, 2, 1)
43      private int turno;
44      private int direccion = 1;
45      // Si el jugador en turno ya robo del mazo en este turno. Se puede seguir robando,
46      // pero desde el primer robo ya se permite pasar (regla oficial).
47      private bool yaRobo;
48
49      // "Decir UNO": al tirar la penultima carta, el jugador tiene unos segundos para
50      // presionar el boton; si no lo hace, roba 2 cartas de castigo.
51      private const int SegundosParaUno = 4;
52      private Button botonUno;
53      private readonly System.Windows.Forms.Timer temporizadorUno = new
    System.Windows.Forms.Timer { Interval = 1000 };
54      private int segundosRestantesUno;
55      // Posicion del jugador que debe decir UNO (-1 = nadie). Mientras corre el tiempo, el
    juego se pausa.
56      private int jugadorEnUno = -1;
57
58      // Controles de cada jugador, en orden de posicion:
59      // 1 abajo, 2 derecha, 3 arriba, 4 izquierda
60      private Label[] etiquetasNombre;
61      private Label[] indicadoresTurno;
62      private PictureBox pictureBoxDescarte;
63      private PictureBox pictureBoxMazo;
64      private FlowLayoutPanel[] manosJugadores;
65      // Espacio maximo de cada mano; el panel se ajusta y centra dentro de el
66      private Rectangle[] areasMano = new Rectangle[NumJugadores];
67
68      // Las imagenes se cargan una sola vez y se reutilizan (cargarlas en cada jugada
69      // deja archivos bloqueados y consume memoria)
70      private readonly Dictionary<string, Image> imagenes = new Dictionary<string, Image>();
```

Constantes (líneas 17 a 28):

- Líneas 17 a 18: 4 jugadores, 7 cartas cada uno.
- Líneas 21 a 22: cada carta mide 80×120 px. Línea 24 al pasar el mouse "sube" 12 px. Línea 26 mazo y descarte miden 100×150.
- Línea 28: un solo generador de números aleatorios para todo el juego (crear varios seguidos puede dar la misma secuencia).

Estado de la partida (líneas 31 a 47), esto es lo que "sabe" el juego en cada momento:

Variable	Línea	Significado
`jugadores`	31	Los 4 jugadores ordenados por posición; el índice 0 es el jugador de abajo.
`mazo`	32	Cartas que quedan para robar. `mazo[0]` es la de arriba.
`cartaArriba`	34	La carta visible en el descarte.
`colorActual`	37	El color que vale. Normalmente el de `cartaArriba`, pero después de un comodín es el elegido. Se guarda aparte porque no se modifica la carta (el comodín no tiene color propio).
`idPartida`	38	Id de la partida en la base de datos.
`partidaEnCurso`	39	Hay una partida activa.
`turno`	43	Índice (0–3) del jugador que tiene el turno.
`direccion`	44	1 = 1 → 2 → 3 → 4, −1 = al revés. Cambia con la reversa.
`yaRobo`	47	Si el jugador en turno ya robó; habilita "Pasar".

Cuenta de UNO (líneas 51 a 56): 4 segundos para presionar el botón, el botón, un `Timer` que hace "tic" cada 1000 ms, los segundos que quedan y `jugadorEnUno` (−1 = nadie). Mientras `jugadorEnUno >= 0` el juego está pausado.

Controles (líneas 60 a 66): arreglos para manejar los 4 jugadores con un índice: etiquetas de nombre, indicadores ●, paneles de mano, y `areasMano` (el rectángulo máximo que puede ocupar cada mano).

Caché de imágenes (línea 70): diccionario `nombre de archivo|giro → Image`. Cada imagen se carga una sola vez.

Posiciones en pantalla: índice 0 = abajo (`flowMano1`), 1 = derecha (`flowMano2`), 2 = arriba (`flowMano3`), 3 = izquierda (`flowMano4`). El índice de `jugadores` coincide con el de todos los arreglos de controles.

5.2 Constructor Ventana() (líneas 72–93)

Fragmento de Form1.cs (líneas 72 a 93)

```
72  public Ventana()
73  {
74      InitializeComponent();
75
76      etiquetasNombre = new[] { lblJugador1, lblJugador2, lblJugador3, lblJugador4 };
77      indicadoresTurno = etiquetasNombre.Select(_ => CrearIndicadorTurno()).ToArray();
78
79      pictureBoxDescarte = pictureBox4;
80      pictureBoxMazo = pictureBox6;
81      manosJugadores = new[] { flowMano1, flowMano2, flowMano3, flowMano4 };
82
83      AplicarEstilo();
```

```
84      pictureBoxMazo.Click += (s, e) => RobarCarta(turno);
85      pictureBoxMazo.Cursor = Cursors.Hand;
86      temporizadorUno.Tick += (s, e) => AvanzarCuentaUno();
87
88      ActualizarTurno();
89      Load += (s, e) => AcomodarLayout();
90      Resize += (s, e) => AcomodarLayout();
91
92      FormClosing += Ventana_FormClosing;
93  }
```

- Línea 74: `InitializeComponent()` crea los controles definidos en `Form1.Designer.cs`. Siempre va primero.
- Línea 76: junta las 4 etiquetas en un arreglo. Línea 77 crea un indicador ● por cada etiqueta (`Select` sobre las 4, ignorando el valor con _).
- Líneas 79 a 81: da nombres claros a los controles del diseñador: `pictureBox4` es el descarte, `pictureBox6` es el mazo.
- Línea 83: aplica colores y fuentes.
- Línea 84: clic en el mazo → `RobarCarta(turno)`: siempre roba quien tiene el turno.
- Línea 86: cada segundo del timer → `AvanzarCuentaUno()`.
- Línea 88: dibuja el estado inicial (sin partida).
- Líneas 89 a 90: al cargar y al cambiar el tamaño de la ventana se reacomoda la mesa.
- Línea 92: al cerrar se ejecuta `Ventana_FormClosing`.

5.3 AplicarEstilo (líneas 98–160)

Fragmento de Form1.cs (líneas 98 a 160)

```
98  private void AplicarEstilo()
99  {
100     // Evita el parpadeo al redibujar las cartas sobre la imagen de fondo
101     DoubleBuffered = true;
102
103     for (int i = 0; i < NumJugadores; i++)
104     {
105         var panel = manosJugadores[i];
106         ActivarDobleBuffer(panel);
107         panel.AutoScroll = true;
108         panel.WrapContents = false; // una sola fila (o columna) con scroll
109         panel.FlowDirection = EsLateral(i) ? FlowDirection.TopDown :
    FlowDirection.LeftToRight;
110         panel.BackColor = Color.Transparent;
111
112         var etiqueta = etiquetasNombre[i];
113         etiqueta.Font = new Font("Segoe UI", 11, FontStyle.Bold);
114         etiqueta.ForeColor = Color.White;
115         etiqueta.BackColor = Color.FromArgb(140, 0, 0, 0);
116         etiqueta.Padding = new Padding(8, 3, 8, 3);
117     }
118
119     foreach (var pb in new[] { pictureBoxDescarte, pictureBoxMazo })
120     {
```

```
121         pb.Size = TamañoCentro;
122         pb.SizeMode = PictureBoxSizeMode.Zoom;
123         pb.BackColor = Color.Transparent;
124     }
125 
126     // label1 indica el color en juego (importa sobre todo despues de un comodin)
127     label1.Text = "";
128     label1.Visible = false;
129     label1.Font = new Font("Segoe UI", 11, FontStyle.Bold);
130     label1.Padding = new Padding(10, 4, 10, 4);
131 
132     botonPasar.FlatStyle = FlatStyle.Flat;
133     botonPasar.FlatAppearance.BorderColor = Color.White;
134     botonPasar.BackColor = Color.FromArgb(200, 20, 20, 20);
135     botonPasar.ForeColor = Color.White;
136     botonPasar.Font = new Font("Segoe UI", 10, FontStyle.Bold);
137     botonPasar.Cursor = Cursors.Hand;
138     botonPasar.AutoSize = true;
139     botonPasar.Padding = new Padding(8, 2, 8, 2);
140 
141     // Boton "¡UNO!": amarillo con borde negro, como el logo. Solo aparece durante la
    cuenta.
142     botonUno = new Button
143     {
144         Text = "¡UNO!",
145         FlatStyle = FlatStyle.Flat,
146         BackColor = Color.Gold,
147         ForeColor = Color.Black,
148         Font = new Font("Segoe UI", 16, FontStyle.Bold),
149         AutoSize = true,
150         Padding = new Padding(14, 4, 14, 4),
151         Cursor = Cursors.Hand,
152         Visible = false
153     };
154     botonUno.FlatAppearance.BorderColor = Color.Black;
155     botonUno.FlatAppearance.BorderSize = 3;
156     botonUno.FlatAppearance.MouseOverBackColor = Color.Yellow;
157     botonUno.Click += (s, e) => DecirUno();
158     Controls.Add(botonUno);
159     botonUno.BringToFront();
160 }
```

El equipo decidió crear los estilos desde código y no desde el diseñador, porque `Form1.Designer.cs` causaba conflictos al hacer merge en Git.

- Línea 101: `DoubleBuffered` dibuja primero en memoria y luego en pantalla: evita parpadeo.
- Líneas 103 a 117: para cada jugador: activa doble buffer en su panel (línea 106), scroll (línea 107), una sola fila sin saltos de línea (línea 108), dirección vertical en los lados y horizontal arriba/abajo (línea 109), fondo transparente. La etiqueta del nombre en blanco sobre negro semitransparente (`FromArgb(140, 0,0,0)`: 140 es la opacidad de 0 a 255).
- Líneas 119 a 124: mazo y descarte: tamaño 100×150 y `Zoom` (la imagen se escala sin deformarse).
- Líneas 127 a 130: `label1` mostrará el color en juego; empieza oculta.

- Líneas 132 a 139: estilo plano para el botón "Pasar".
- Líneas 142 a 159: crea el botón ¡UNO! desde código: dorado con borde negro, invisible al inicio. Línea 157 al hacer clic llama a `DecirUno()`. Líneas 158 a 159 se agrega a la ventana y se pone al frente.

5.4 CrearIndicadorTurno (líneas 165–179)

Fragmento de Form1.cs (líneas 165 a 179)

```
165 private Label CrearIndicadorTurno()
166 {
167     var indicador = new Label
168     {
169         Text = "●",
170         ForeColor = Color.Yellow,
171         BackColor = Color.Transparent,
172         AutoSize = true,
173         Font = new Font(Font.FontFamily, 14, FontStyle.Bold),
174         Visible = false
175     };
176     Controls.Add(indicador);
177     indicador.BringToFront();
178     return indicador;
179 }
```

Crea una etiqueta con el carácter "●" amarillo, invisible. Se muestra junto al nombre del jugador que tiene el turno (ver `ActualizarTurno`).

5.5 EmpezarPartida (líneas 184–242)

Fragmento de Form1.cs (líneas 184 a 242)

```
184 /// <summary>
185 /// Pide los nombres, reparte y registra la partida en la base de datos.
186 /// Devuelve false si se cancelo la ventana de nombres o fallo la API.
187 /// La usa el boton "Empezar juego" y tambien la pantalla de inicio (FormInicio).
188 /// </summary>
189 public bool EmpezarPartida()
190 {
191     // Se sugieren los nombres de la partida anterior, si la hubo
192     List<string> nombres;
193     using (var dialogo = new FormNombres(NumJugadores, jugadores.Select(j =>
    j.Nombre).ToList()))
194     {
195         if (dialogo.ShowDialog(this) != DialogResult.OK)
196             return false;
197         nombres = dialogo.Nombres;
198     }
199
200     try
201     {
202         // Si habia una partida sin terminar, queda abandonada
203         if (partidaEnCurso)
204         {
205             BaseDatos.TerminarPartida(idPartida, null);
206             partidaEnCurso = false;
207         }
```

```
208
209         jugadores = nombres.Select(BaseDatos.ObtenerOCrearJugador).ToList();
210
211         mazo = BaseDatos.ObtenerCartas();
212         Barajar(mazo);
213         Repartir();
214
215         cartaArriba = SacarCartaInicial();
216         colorActual = cartaArriba.Color;
217
218         idPartida = BaseDatos.RegistrarPartida(jugadores, cartaArriba);
219         partidaEnCurso = true;
220
221         // El orden de los turnos sale de la posicion guardada en partida_jugador
222         var posiciones = BaseDatos.ObtenerPosiciones(idPartida);
223         foreach (var jugador in jugadores)
224             jugador.Posicion = posiciones[jugador.Id];
225         jugadores = jugadores.OrderBy(j => j.Posicion).ToList();
226     }
227     catch (ErrorBaseDatos ex)
228     {
229         MostrarErrorBaseDatos(ex);
230         return false;
231     }
232
233     // La carta inicial siempre es de numero: empieza la posicion 1 en sentido 1→2→3→4
234     direccion = 1;
235     turno = 0;
236     yaRobo = false;
237     DetenerCuentaUno();
238     MostrarManos();
239     MostrarDescarte();
240     ActualizarTurno();
241     return true;
242 }
```

Es pública porque también la llama `FormInicio`. Devuelve `true` si la partida empezó.

- Líneas 193 a 198: abre `FormNombres`. Le pasa los nombres de la partida anterior para sugerirlos. Si se cancela, regresa `false`. `using` libera la ventana al cerrarla.
- Líneas 203 a 207: si había una partida en curso, se termina sin ganador = abandonada. Antes esto pasaba al presionar el botón "Empezar juego" a la mitad de una partida; desde que Joaquin quitó ese botón (commit `a1f83dc`), cada partida se juega en una ventana nueva, así que en la práctica ya casi no se ejecuta.
- Línea 209: `nombres.Select(BaseDatos.ObtenerOCrearJugador)`: llama a la API una vez por nombre y obtiene 4 objetos `Jugador` con su id.
- Líneas 211 a 213: pide las 60 cartas, las baraja y reparte.
- Líneas 215 a 216: voltea la carta inicial y su color pasa a ser el color en juego.
- Líneas 218 a 219: registra todo en la base (una sola petición) y marca la partida en curso.
- Líneas 222 a 225: pide a la API la posición de cada jugador, la asigna y ordena la lista por posición. Así el orden de turnos es el que quedó guardado en `partida_jugador`.

- Líneas 227 a 231: si cualquier llamada a la API falla, muestra el error y regresa `false`.
- Líneas 234 a 241: reinicia dirección, turno, `yaRobo` y la cuenta de UNO; dibuja manos, descarte y turno.

5.6 JugarCarta: la regla principal (líneas 244–303)

Se ejecuta con doble clic en una carta (ver `MostrarMano` línea 599).

Fragmento de Form1.cs (líneas 244 a 303)

```
244 private void JugarCarta(int posicion, Carta carta, PictureBox pic)
245 {
246     // Solo puede jugar quien tiene el turno, y no mientras alguien debe decir UNO
247     if (posicion != turno || !partidaEnCurso || jugadorEnUno >= 0)
248         return;
249 
250     var jugador = jugadores[posicion];
251 
252     bool esComodin = (carta.Efecto != null && (carta.Efecto.Equals("cambiar_color") ||
    carta.Efecto.Equals("+4")));
253     bool coincideColor = (carta.Color != null && carta.Color.Equals(colorActual));
254     bool coincideNumero = (carta.Numero != null && carta.Numero == cartaArriba.Numero);
255     bool coincideEfecto = (carta.Efecto != null && carta.Efecto.Equals(cartaArriba.Efecto));
256 
257     if (!esComodin && !coincideColor && !coincideNumero && !coincideEfecto)
258     {
259         //aqui va el cambio de color por milisegundos
260         EfectoCartaInvalida(pic);
261         return;
262     }
263 
264     string colorElegido = null;
265     if (esComodin)
266     {
267         colorElegido = PedirColor();
268         if (colorElegido == null)
269             return;
270     }
271 
272     try
273     {
274         BaseDatos.RegistrarJugada(idPartida, jugador.Id, "tirar", carta.Id, colorElegido);
275     }
276     catch (ErrorBaseDatos ex)
277     {
278         MostrarErrorBaseDatos(ex);
279         return;
280     }
281 
282     jugador.Mano.Remove(carta);
283     RegresarAlMazo(cartaArriba);
284     MostrarMano(posicion);
285 
286     if (jugador.Mano.Count == 0)
287         TerminarConGanador(jugador);
288     else
289         AplicarEfecto(carta);
290     yaRobo = false; // el turno paso al siguiente jugador
291     cartaArriba = carta;
```

```
292     colorActual = colorElegido ?? carta.Color;
293     MostrarDescarte();
294     ActualizarTurno();
295 
296     // Tiro su penultima carta: tiene unos segundos para decir UNO
297     if (partidaEnCurso && jugador.Mano.Count == 1)
298         IniciarCuentaUno(posicion);
299 
300     // Se quedo sin cartas y la partida quedo registrada: anuncio y de vuelta al inicio
301     if (jugador.Mano.Count == 0 && !partidaEnCurso)
302         AnunciarGanador(jugador);
303 }
```

1) Bloqueos (líneas 247 a 248): no hace nada si no es el turno de ese jugador, si no hay partida o si alguien está en la cuenta de UNO.

2) Validación (líneas 252 a 262): la carta se puede jugar si cumple al menos una condición:

Variable	Línea	Condición
`esComodin`	252	Es `cambiar_color` o `+4`: se puede jugar siempre.
`coincideColor`	253	Su color es igual a `colorActual` (no al de la carta de arriba, por si hubo comodín).
`coincideNumero`	254	Tiene número y es igual al de la carta de arriba.
`coincideEfecto`	255	Mismo símbolo (ej. +2 sobre +2, bloqueo sobre bloqueo), aunque sean de distinto color.

Si no cumple ninguna, llama a `EfectoCartaInvalida(pic)` (línea 260), que pinta la carta de rojo por un momento, y sale sin hacer nada más. Por eso `JugarCarta` recibe también el `PictureBox` de la carta (`pic`, línea 244). Antes aquí se mostraba un `MessageBox` con el texto "No puedes jugar...".

Nota: En línea 254, si las dos cartas son especiales, `carta.Numero != null` es falso y la condición no se cumple aunque ambos números sean null. Por eso es necesaria la revisión de `null` antes de comparar.

3) Color del comodín (líneas 264 a 270): si es comodín abre `PedirColor()`. Si el usuario cierra sin elegir (`null`), no se juega nada.

4) Registrar primero (líneas 272 a 280): se guarda `tirar` en la base antes de cambiar el estado del juego. Si la API falla, se sale sin haber movido nada: el juego y la base no quedan desincronizados.

5) Mover la carta (líneas 282 a 284): se quita de la mano; la carta que estaba arriba regresa al mazo en un lugar aleatorio (así el mazo nunca se acaba rápido); se redibuja la mano.

6) Ganador o efecto (líneas 286 a 289): si la mano quedó vacía, `TerminarConGanador`; si no, `AplicarEfecto` mueve el turno.

7) Actualizar estado (líneas 290 a 294): `yaRobo = false` porque el turno ya cambió; la carta jugada pasa a ser `cartaArriba`; `colorActual = colorElegido ?? carta.Color` (si eligió color se usa ese, si no, el de la carta); se redibuja.

8) UNO (líneas 297 a 298): si le queda 1 carta, empieza la cuenta para decir UNO.

9) Anuncio (líneas 301 a 302): si quedó sin cartas y la partida ya se marcó terminada, muestra al ganador.

5.7 EfectoCartaInvalida (líneas 305–344)

Lo agregó Joaquin (commit `bb4ea85`) para que una jugada inválida no interrumpa el juego con una ventana de mensaje.

Fragmento de Form1.cs (líneas 305 a 344)

```
305 private async void EfectoCartaInvalida(PictureBox pic)
306 {
307     // Si la carta ya está bloqueada/animándose, ignoramos clics extra
308     if (!pic.Enabled) return;
309 
310     // Desactivamos la carta temporalmente para evitar spam de clics
311     pic.Enabled = false;
312 
313     // Creamos un evento "Paint" temporal.
314     // Esto nos permite dibujar SOBRE la imagen de la carta.
315     PaintEventHandler filtroRojo = (sender, e) =>
316     {
317         // Color rojo con opacidad (120 de 255)
318         using (Brush brocha = new SolidBrush(Color.FromArgb(120, 255, 0, 0)))
319         {
320             e.Graphics.FillRectangle(brocha, pic.ClientRectangle);
321         }
322     };
323 
324     try
325     {
326         // Agregamos el filtro y forzamos a que el PictureBox se redibuje
327         pic.Paint += filtroRojo;
328         pic.Invalidate();
329 
330         // Esperamos 350 milisegundos (puedes ajustar este tiempo)
331         await Task.Delay(350);
332 
333         if (!pic.IsDisposed)
334         {
335             pic.Paint -= filtroRojo;
336             pic.Enabled = true;
337             pic.Invalidate();
338         }
339     }
340     catch
341     {
342         // Se ignora silenciosamente si el usuario cerró el juego a mitad del parpadeo
343     }
344 }
```

- Línea 305: es `async void`: puede esperar con `await` sin congelar la ventana. Se usa `void` (y no `Task`) porque se llama como si fuera un evento, sin esperar su resultado.
- Línea 308: si la carta ya está desactivada (porque se está animando), no hace nada. Así, si el jugador hace doble clic muchas veces, el efecto no se acumula.
- Línea 311: desactiva la carta mientras dura el efecto.
- Líneas 315 a 322: crea un manejador del evento `Paint` guardado en la variable `filtroRojo`. Cada vez que el `PictureBox` se dibuja, este manejador pinta encima un rectángulo rojo semitransparente (`FromArgb(120, 255, 0, 0)`: 120 de opacidad).
- Líneas 327 a 328: agrega el manejador al evento y llama a `Invalidate()` para que la carta se redibuje ya con el rojo.
- Línea 331: `await Task.Delay(350)` espera 350 milisegundos sin bloquear el programa.
- Líneas 333 a 338: si la carta todavía existe (`IsDisposed` es falso), quita el manejador (`-=`), vuelve a activar la carta y la redibuja sin el rojo.
- Líneas 340 a 343: si algo falla (por ejemplo, se cerró el juego durante esos 350 ms), el error se ignora.

5.8 botonPasar_Click (líneas 350–368)

Fragmento de Form1.cs (líneas 350 a 368)

```
350 private void botonPasar_Click(object sender, EventArgs e)
351 {
352     if (!partidaEnCurso || jugadorEnUno >= 0)
353         return;
354 
355     try
356     {
357         BaseDatos.RegistrarJugada(idPartida, jugadores[turno].Id, "pasar", null);
358     }
359     catch (ErrorBaseDatos ex)
360     {
361         MostrarErrorBaseDatos(ex);
362         return;
363     }
364 
365     yaRobo = false;
366     turno = Siguiente(turno);
367     ActualizarTurno();
368 }
```

Registra `pasar` (sin carta: `null`), reinicia `yaRobo` y pasa el turno al siguiente. El botón solo es visible después de robar o con el mazo vacío (ver `ActualizarTurno` líneas 675 a 676).

5.9 PedirColor (líneas 373–383)

Fragmento de Form1.cs (líneas 373 a 383)

```
373 private string PedirColor()
374 {
375     using (FormElegirColor ventanaElegirColor = new FormElegirColor())
376     {
377         if (ventanaElegirColor.ShowDialog(this) != DialogResult.OK)
378             return null;
379 
380         string nuevoColor = ventanaElegirColor.getColor();
381         return nuevoColor;
382     }
383 }
```

Abre `FormElegirColor` como ventana modal (`ShowDialog`: bloquea la mesa hasta cerrarla). Si se eligió color (`DialogResult.OK`) lo devuelve; si no, `null`.

5.10 AplicarEfecto (líneas 388–413)

Fragmento de Form1.cs (líneas 388 a 413)

```
388 private void AplicarEfecto(Carta carta)
389 {
390     switch (carta.Efecto)
391     {
392         case "cambiar_direccion":
393             direccion = -direccion;
394             turno = Siguiente(turno);
395             break;
396 
397         case "bloquear":
398             turno = Siguiente(turno, 2);
399             break;
400 
401         case "+2":
402             DarCartas(Siguiente(turno), 2);
403             turno = Siguiente(turno, 2);
404             break;
405         case "+4":
406             DarCartas(Siguiente(turno), 4);
407             turno = Siguiente(turno, 2);
408             break;
409         default:
410             turno = Siguiente(turno);
411             break;
412     }
413 }
```

Efecto	Qué hace	Por qué
`cambiar_direccion` (reversa)	invierte `direccion` y avanza 1	Primero se cambia el sentido, luego se avanza en el nuevo sentido.
`bloquear`	avanza 2	Se salta al siguiente jugador.
`+2`	da 2 cartas al siguiente y avanza 2	El castigado roba y además pierde el turno.

Efecto	Qué hace	Por qué
`+4`	da 4 cartas al siguiente y avanza 2	Igual que el +2 pero con 4 cartas.
cualquier otra (número o cambio de color)	avanza 1	Turno normal.

5.11 Siguiente: el cálculo del turno (líneas 418–422)

Fragmento de Form1.cs (líneas 418 a 422)

```
418 private int Siguiente(int desde, int pasos = 1)
419 {
420     int n = jugadores.Count;
421     return ((desde + direccion * pasos) % n + n) % n;
422 }
```

Calcula quién juega después, dando la vuelta a la mesa con el operador módulo (`%`, el residuo de la división).

- `desde + direccion * pasos` puede salirse del rango 0–3 (ej. 3 + 1 = 4, o 0 − 1 = −1).
- En C#, `%` con números negativos da negativo (−1 % 4 = −1). Por eso se hace `(x % n + n) % n`: siempre queda entre 0 y n−1.

desde	direccion	pasos	Cálculo	Resultado
3	1	1	((4 % 4) + 4) % 4	0
0	−1	1	((−1 % 4) + 4) % 4 = (−1 + 4) % 4	3
1	1	2	((3 % 4) + 4) % 4	3
3	1	2	((5 % 4) + 4) % 4	1

`pasos = 1` es un parámetro con valor por defecto: `Siguiente(turno)` avanza 1.

5.12 DarCartas (líneas 429–462)

Fragmento de Form1.cs (líneas 429 a 462)

```
429 private int DarCartas(int posicion, int cantidad, string accion = "robar")
430 {
431     var jugador = jugadores[posicion];
432     int dadas = 0;
433 
434     for (int i = 0; i < cantidad; i++)
435     {
436         if (mazo.Count == 0)
437         {
438             MessageBox.Show("Ya no quedan cartas para robar.", "UNO",
439                 MessageBoxButtons.OK, MessageBoxIcon.Information);
440             break;
441         }
442 
443         var carta = mazo[0];
```

```
444         try
445         {
446             BaseDatos.RegistrarJugada(idPartida, jugador.Id, accion, carta.Id);
447         }
448         catch (ErrorBaseDatos ex)
449         {
450             MostrarErrorBaseDatos(ex);
451             break;
452         }
453 
454         mazo.RemoveAt(0);
455         jugador.Mano.Add(carta);
456         dadas++;
457     }
458 
459     if (dadas > 0)
460         MostrarMano(posicion);
461     return dadas;
462 }
```

Se usa para robar (1 carta), para el +2 y +4, y para el castigo de UNO (`accion = "penalizacion_uno"`).

- Línea 434: repite `cantidad` veces. Líneas 436 a 441 si el mazo está vacío avisa y para.
- Líneas 443 a 452: toma la carta de arriba (`mazo[0]`) y primero la registra en la base; si falla, para.
- Líneas 454 a 456: la quita del mazo y la agrega a la mano.
- Líneas 459 a 461: redibuja solo si dio alguna carta y devuelve cuántas dio.

5.13 Fin de la partida (líneas 464–506)

Fragmento de Form1.cs (líneas 464 a 506)

```
464 private void TerminarConGanador(Jugador ganador)
465 {
466     try
467     {
468         BaseDatos.TerminarPartida(idPartida, ganador.Id);
469     }
470     catch (ErrorBaseDatos ex)
471     {
472         MostrarErrorBaseDatos(ex);
473         return;
474     }
475 
476     partidaEnCurso = false;
477     ActualizarTurno();
478 }
479 
480 /// <summary>
481 /// Muestra el anuncio del ganador; al aceptar se cierra la mesa y se regresa
482 /// a la pantalla de inicio (FormInicio vuelve a aparecer al cerrar el juego).
483 /// </summary>
484 private void AnunciarGanador(Jugador ganador)
485 {
486     using (var anuncio = new FormGanador(ganador.Nombre, idPartida))
487         anuncio.ShowDialog(this);
```

```
488     Close();
489 }
490 
491 // Al cerrar el programa, la partida en curso queda terminada sin ganador
492 private void Ventana_FormClosing(object sender, FormClosingEventArgs e)
493 {
494     DetenerCuentaUno();
495     if (!partidaEnCurso)
496         return;
497 
498     try
499     {
500         BaseDatos.TerminarPartida(idPartida, null);
501     }
502     catch (ErrorBaseDatos ex)
503     {
504         MostrarErrorBaseDatos(ex);
505     }
506 }
```

- TerminarConGanador (líneas 464 a 478): `PUT /terminar` con el id del ganador; marca `partidaEnCurso = false`.
- AnunciarGanador (líneas 484 a 489): muestra `FormGanador` (modal). Al cerrarlo, `Close()` cierra la mesa y se vuelve a FormInicio.
- Ventana_FormClosing (líneas 492 a 506): al cerrar la mesa detiene la cuenta de UNO y, si había partida en curso, la termina sin ganador (abandonada).

5.14 Barajar, Repartir, carta inicial y mazo (líneas 511–563)

Fragmento de Form1.cs (líneas 511 a 563)

```
511 private static void Barajar(List<Carta> cartas)
512 {
513     for (int i = cartas.Count - 1; i > 0; i--)
514     {
515         int j = aleatorio.Next(i + 1);
516         var temporal = cartas[i];
517         cartas[i] = cartas[j];
518         cartas[j] = temporal;
519     }
520 }
521 
522 /// <summary>
523 /// Reparte una carta a cada jugador por vuelta, tomandolas del tope del mazo.
524 /// Las cartas que sobran se quedan en el mazo para robar.
525 /// </summary>
526 private void Repartir()
527 {
528     for (int vuelta = 0; vuelta < CartasPorJugador; vuelta++)
529     {
530         foreach (var jugador in jugadores)
531         {
532             jugador.Mano.Add(mazo[0]);
533             mazo.RemoveAt(0);
534         }
535     }
536 }
537 
```

```
538 /// <summary>
539 /// Voltea la carta de arriba del mazo para empezar el juego.
540 /// Solo puede ser una carta ordinaria (numero): si sale un comodin o una especial,
541 /// se regresa al mazo y se voltea otra. Siempre quedan ordinarias en el mazo
542 /// despues de repartir (hay 36 ordinarias y solo 24 especiales).
543 /// </summary>
544 private Carta SacarCartaInicial()
545 {
546     var carta = mazo[0];
547     while (carta.Tipo != "ordinaria")
548     {
549         mazo.RemoveAt(0);
550         mazo.Insert(aleatorio.Next(1, mazo.Count + 1), carta);
551         carta = mazo[0];
552     }
553     mazo.RemoveAt(0);
554     return carta;
555 }
556
557 /// <summary>
558 /// Mete la carta al mazo en una posicion aleatoria, para que robar siga siendo al azar.
559 /// </summary>
560 private void RegresarAlMazo(Carta carta)
561 {
562     mazo.Insert(aleatorio.Next(mazo.Count + 1), carta);
563 }
```

Barajar (líneas 511 a 520): algoritmo Fisher-Yates. Recorre la lista desde el final; en cada paso elige una posición al azar `j` entre 0 e `i` (línea 515) e intercambia los elementos `i` y `j` (líneas 516 a 518). Así cada orden posible tiene la misma probabilidad. Es `static` porque no usa nada del objeto.

Repartir (líneas 526 a 536): 7 vueltas; en cada una, una carta a cada jugador (como se reparte en la vida real). Se toma `mazo[0]` y se elimina del mazo. Después de repartir quedan 60 − 28 = 32 cartas.

SacarCartaInicial (líneas 544 a 555): la carta inicial debe ser ordinaria. Mientras la de arriba sea especial (línea 547), la quita y la vuelve a meter en una posición aleatoria entre 1 y el final (línea 550; nunca en 0, para no sacarla otra vez enseguida). Siempre termina porque hay 36 ordinarias. Al final la quita del mazo (línea 553): quedan 31.

RegresarAlMazo (líneas 560 a 563): inserta la carta en una posición aleatoria del mazo, para que robar siga siendo al azar.

5.15 Dibujar las manos (líneas 565–624)

Fragmento de Form1.cs (líneas 565 a 624)

```
565 private void MostrarManos()
566 {
567     for (int i = 0; i < jugadores.Count; i++)
568         MostrarMano(i);
569 }
570
```

```
571 /// <summary>
572 /// Dibuja todas las cartas de la mano del jugador, boca arriba, dentro de su
573 /// FlowLayoutPanel. Doble clic en una carta para jugarla. Las cartas del jugador
574 /// en turno se elevan al pasar el mouse sobre ellas.
575 /// </summary>
576 private void MostrarMano(int posicion)
577 {
578     var contenedor = manosJugadores[posicion];
579     var giro = GiroCartas(posicion);
580     var tamaño = EsLateral(posicion) ? new Size(AltoCarta, AnchoCarta) : new
    Size(AnchoCarta, AltoCarta);
581
582     contenedor.SuspendLayout();
583
584     // Dispose libera los PictureBox anteriores; Controls.Clear() solo los quitaria
585     foreach (var anterior in contenedor.Controls.Cast<Control>().ToList())
586         anterior.Dispose();
587
588     foreach (var carta in jugadores[posicion].Mano)
589     {
590         var pic = new PictureBox
591         {
592             Image = ObtenerImagenCarta(carta, null, giro),
593             SizeMode = PictureBoxSizeMode.Zoom,
594             Size = tamaño,
595             Margin = MargenCarta(posicion, false),
596             BackColor = Color.Transparent,
597             Tag = carta
598         };
599         pic.DoubleClick += (s, e) => JugarCarta(posicion, carta, pic);
600         pic.MouseEnter += (s, e) =>
601         {
602             if (partidaEnCurso && posicion == turno)
603                 pic.Margin = MargenCarta(posicion, true);
604         };
605         pic.MouseLeave += (s, e) => pic.Margin = MargenCarta(posicion, false);
606         contenedor.Controls.Add(pic);
607     }
608
609     contenedor.ResumeLayout();
610
611     ActualizarEtiqueta(posicion);
612 }
613
614 /// <summary>
615 /// Nombre del jugador, cuantas cartas tiene y "¡UNO!" si ya lo dijo con su ultima carta.
616 /// </summary>
617 private void ActualizarEtiqueta(int posicion)
618 {
619     int cantidad = jugadores[posicion].Mano.Count;
620     string uno = cantidad == 1 && jugadorEnUno != posicion ? " · ¡UNO!" : "";
621     etiquetasNombre[posicion].Text =
622         $"{jugadores[posicion].Nombre} · {cantidad} {(cantidad == 1 ? "carta" :
    "cartas")}{uno}";
623     AjustarMano(posicion);
624 }
```

MostrarMano (líneas 576 a 612):

- Líneas 578 a 580: obtiene el panel, el giro de la imagen y el tamaño (en los lados la carta está acostada: 120×80).

- Línea 582 / línea 609 `SuspendLayout`/`ResumeLayout`: el panel no se reacomoda con cada carta agregada sino una sola vez al final (más rápido).
- Líneas 585 a 586: elimina con `Dispose` los PictureBox anteriores (libera memoria; `Controls.Clear()` solo los quitaría).
- Líneas 588 a 607: crea un `PictureBox` por carta. `Tag = carta` guarda la carta dentro del control.
- Línea 599: doble clic → `JugarCarta(posicion, carta, pic)`. La lambda "captura" `posicion`, `carta` y el propio `pic` (este último se usa para el efecto rojo).
- Líneas 600 a 605: al entrar el mouse, la carta sube (solo si es su turno); al salir vuelve.

ActualizarEtiqueta (líneas 617 a 624): escribe "Nombre · 5 cartas". Si le queda 1 carta y ya no está en la cuenta (ya dijo UNO o lo castigaron) agrega "· ¡UNO!". Usa singular/plural con un operador ternario (`? :`). Después ajusta el tamaño de la mano.

5.16 Descarte, turno y título (líneas 629–703)

Fragmento de Form1.cs (líneas 629 a 698)

```
629 private void MostrarDescarte()
630 {
631     MostrarCartaEnPictureBox(pictureBoxDescarte, cartaArriba, colorActual);
632 
633     label1.Text = "Color: " + Capitalizar(colorActual);
634     switch (colorActual)
635     {
636         case "rojo": label1.BackColor = Color.DarkRed; label1.ForeColor = Color.White;
    break;
637         case "azul": label1.BackColor = Color.RoyalBlue; label1.ForeColor = Color.White;
    break;
638         case "verde": label1.BackColor = Color.ForestGreen; label1.ForeColor = Color.White;
    break;
639         case "amarillo": label1.BackColor = Color.Gold; label1.ForeColor = Color.Black;
    break;
640         default: label1.BackColor = Color.Transparent; label1.ForeColor = Color.White;
    break;
641     }
642     label1.Visible = true;
643     CentrarDebajo(label1, pictureBoxDescarte);
644 }
645 
646 /// <summary>
647 /// Resalta al jugador en turno con el circulo amarillo y su nombre en amarillo.
648 /// Solo sus cartas muestran la mano para jugar.
649 /// "Pasar" se activa despues de robar (o si el mazo esta vacio).
650 /// </summary>
651 private void ActualizarTurno()
652 {
653     for (int i = 0; i < NumJugadores; i++)
654     {
655         bool enTurno = partidaEnCurso && i == turno;
656         var etiqueta = etiquetasNombre[i];
657         etiqueta.ForeColor = enTurno ? Color.Yellow : Color.White;
658         indicadoresTurno[i].Visible = enTurno;
```

```
659
660         if (enTurno)
661         {
662             indicadoresTurno[i].Location = new Point(
663                 etiqueta.Right + 4,
664                 etiqueta.Top + (etiqueta.Height - indicadoresTurno[i].Height) / 2);
665         }
666
667         foreach (Control carta in manosJugadores[i].Controls)
668         {
669             carta.Cursor = enTurno ? Cursors.Hand : Cursors.Default;
670             carta.Margin = MargenCarta(i, false);
671         }
672     }
673
674     // Solo se muestra cuando se puede usar, para no estorbar junto al mazo
675     botonPasar.Enabled = partidaEnCurso && jugadorEnUno < 0 && (yaRobo || mazo.Count == 0);
676     botonPasar.Visible = botonPasar.Enabled;
677
678     ActualizarTitulo();
679 }
680
681 private void ActualizarTitulo()
682 {
683     if (!partidaEnCurso)
684     {
685         Text = idPartida == 0 ? "UNO" : $"UNO - Partida {idPartida} terminada";
686         return;
687     }
688
689     string sentido = direccion == 1 ? "1→2→3→4" : "4→3→2→1";
690     Text = $"UNO - Partida {idPartida} | Turno de {jugadores[turno].Nombre} | " +
691            $"Sentido {sentido} | {mazo.Count} cartas en el mazo";
692 }
693
694 private static void MostrarErrorBaseDatos(ErrorBaseDatos ex)
695 {
696     MessageBox.Show("Error con la base de datos:\n" + ex.Message,
697         "UNO", MessageBoxButtons.OK, MessageBoxIcon.Error);
698 }
```

- MostrarDescarte (líneas 629 a 644): pone la imagen de la carta de arriba y escribe "Color en juego: Rojo" en `label1` con el fondo del color (`switch` líneas 634 a 641). Amarillo usa texto negro para que se lea.
- ActualizarTurno (líneas 651 a 679): para cada jugador: nombre amarillo y ● visible si tiene el turno (líneas 655 a 665, el ● se coloca a la derecha del nombre). Las cartas del jugador en turno tienen cursor de mano (líneas 667 a 671). Línea 675 "Pasar" se habilita solo si hay partida, nadie está en la cuenta de UNO y (ya robó o el mazo está vacío). Línea 676 si no está habilitado, ni siquiera se ve.
- ActualizarTitulo (líneas 681 a 692): la barra de título muestra partida, turno, sentido y cartas del mazo. Ej.: "UNO - Partida 7 | Turno de Ana | Sentido 1→2→3→4 | 25 cartas en el mazo".
- MostrarErrorBaseDatos (líneas 694 a 698): ventana de error estándar para cualquier `ErrorBaseDatos`.

Observación: `pictureBox4_Click` (líneas 700 a 703) está vacío: es un manejador que generó el diseñador al hacer doble clic sobre el control. No hace nada, pero no se puede borrar sin quitar también la línea 58 de `Form1.Designer.cs`.

5.17 Imágenes de las cartas (líneas 709–768)

Fragmento de Form1.cs (líneas 709 a 768)

```
709 private static string NombreArchivo(Carta carta, string colorMostrar = null)
710 {
711     if (carta.Tipo == "ordinaria")
712         return $"{carta.Color}_{carta.Numero}.png";
713 
714     string color = carta.Color ?? colorMostrar;
715 
716     switch (carta.Efecto)
717     {
718         case "cambiar_color": return "comodin_color.png";
719         case "+4": return "comodin_mas4.png";
720         case "bloquear": return $"{color}_bloqueo.png";
721         case "cambiar_direccion": return $"{color}_reversa.png";
722         case "+2": return $"{color}_mas2.png";
723         default: return "comodin_color.png";
724     }
725 }
726 
727 /// <summary>
728 /// Imagen de la carta, cargada una sola vez y guardada para reutilizarla.
729 /// 'giro' sirve para las manos de los lados, donde las cartas van acostadas.
730 /// </summary>
731 private Image ObtenerImagenCarta(Carta carta, string colorMostrar = null,
732     RotateFlipType giro = RotateFlipType.RotateNoneFlipNone)
733 {
734     string archivo = NombreArchivo(carta, colorMostrar);
735     string clave = archivo + "|" + giro;
736 
737     Image imagen;
738     if (imagenes.TryGetValue(clave, out imagen))
739         return imagen;
740 
741     imagen = CargarImagen(archivo);
742     if (imagen != null && giro != RotateFlipType.RotateNoneFlipNone)
743         imagen.RotateFlip(giro);
744     imagenes[clave] = imagen;
745     return imagen;
746 }
747 
748 /// <summary>
749 /// Lee la imagen de la carpeta Cartas sin dejar el archivo bloqueado.
750 /// Devuelve null si no existe.
751 /// </summary>
752 private static Image CargarImagen(string archivo)
753 {
754     string ruta = Path.Combine(Application.StartupPath, "Cartas", archivo);
755     if (!File.Exists(ruta))
756         return null;
757 
758     using (var flujo = new FileStream(ruta, FileMode.Open, FileAccess.Read))
759     using (var original = Image.FromStream(flujo))
```

```
760         return new Bitmap(original);
761 }
762
763 private void MostrarCartaEnPictureBox(PictureBox pb, Carta carta, string colorMostrar =
    null)
764 {
765     // No se hace Dispose de la imagen anterior: esta guardada para reutilizarse
766     pb.Image = ObtenerImagenCarta(carta, colorMostrar);
767     pb.SizeMode = PictureBoxSizeMode.Zoom;
768 }
```

NombreArchivo (líneas 709 a 725): traduce una carta al nombre de su imagen en la carpeta `Cartas`:

Carta	**Archivo**
Ordinaria roja 5	`rojo_5.png`
Bloqueo azul	`azul_bloqueo.png`
Reversa verde	`verde_reversa.png`
+2 amarillo	`amarillo_mas2.png`
Cambio de color	`comodin_color.png`
+4	`comodin_mas4.png`

ObtenerImagenCarta (líneas 731 a 746): la clave del caché es `archivo|giro` (línea 735) porque la misma carta puede necesitarse derecha o acostada. Si ya está en el diccionario la devuelve (líneas 738 a 739); si no, la carga, la gira si hace falta (líneas 742 a 743) y la guarda.

CargarImagen (líneas 752 a 761): arma la ruta `carpeta del .exe\Cartas\archivo` (línea 754). Si no existe devuelve `null`. Lee el archivo con un `FileStream` y crea una copia con `new Bitmap(original)` (línea 760). Si se usara `Image.FromFile`, Windows dejaría el archivo bloqueado mientras la imagen exista.

MostrarCartaEnPictureBox (líneas 763 a 768): asigna la imagen. No se hace `Dispose` de la anterior porque está en el caché y se va a reutilizar.

5.18 Acomodo de la mesa (líneas 777–913)

Fragmento de Form1.cs (líneas 777 a 807)

```
777 private static bool EsLateral(int posicion)
778 {
779     return posicion == 1 || posicion == 3;
780 }
781
782 private static RotateFlipType GiroCartas(int posicion)
783 {
784     switch (posicion)
785     {
786         case 1: return RotateFlipType.Rotate270FlipNone; // derecha
787         case 3: return RotateFlipType.Rotate90FlipNone; // izquierda
```

```
788         default: return RotateFlipType.RotateNoneFlipNone; // abajo y arriba: derechas para
    leerlas
789     }
790 }
791
792 /// <summary>
793 /// Margen de cada carta. Al elevarla, el espacio se pasa al otro lado para que
794 /// la carta se mueva hacia el centro de la mesa sin mover a las demas.
795 /// </summary>
796 private static Padding MargenCarta(int posicion, bool elevada)
797 {
798     int cerca = elevada ? 0 : Elevacion; // espacio del lado del centro de la mesa
799     int lejos = elevada ? Elevacion : 0;
800     switch (posicion)
801     {
802         case 0: return new Padding(3, cerca, 3, lejos); // abajo: sube
803         case 2: return new Padding(3, lejos, 3, cerca); // arriba: baja
804         case 3: return new Padding(lejos, 3, cerca, 3); // izquierda: va a la derecha
805         default: return new Padding(cerca, 3, lejos, 3); // derecha: va a la izquierda
806     }
807 }
```

- EsLateral (líneas 777 a 780): los jugadores de la derecha (1) y la izquierda (3) tienen las cartas en columna.
- GiroCartas (líneas 782 a 790): derecha → girar 270°, izquierda → 90°, arriba y abajo sin girar.
- MargenCarta (líneas 796 a 807): el margen de cada carta. Normalmente hay 12 px de espacio del lado del centro de la mesa (`cerca`). Al elevarla, ese espacio se pasa al lado de afuera (`lejos`): la carta se mueve hacia el centro sin mover a las demás. `Padding(izq, arriba, der, abajo)`.

Fragmento de Form1.cs (líneas 813 a 856)

```
813 private void AcomodarLayout()
814 {
815     int w = ClientSize.Width;
816     int h = ClientSize.Height;
817     if (w == 0 || h == 0)
818         return; // ventana minimizada
819
820     const int margen = 20;
821     // "Empezar juego" en la esquina superior derecha
822
823
824     int altoEtiqueta = lblJugador1.Height + 6;
825     // Espacio de una fila (arriba/abajo) o columna (lados): carta + elevacion + scroll
826     int grosorMano = AltoCarta + Elevacion + SystemInformation.HorizontalScrollBarHeight +
    6;
827
828     // Abajo y arriba: entre las columnas de los lados
829     int xFila = margen + grosorMano + margen;
830     int anchoFila = Math.Max(AnchoCarta, w - 2 * xFila);
831     int yArriba = margen + altoEtiqueta;
832     int yAbajo = h - margen - grosorMano;
833     areasMano[0] = new Rectangle(xFila, yAbajo, anchoFila, grosorMano);
834     areasMano[2] = new Rectangle(xFila, yArriba, anchoFila, grosorMano);
835
```

```
836     // Lados: casi toda la altura, porque las filas de arriba y abajo no llegan a los
    costados.
837     // Empiezan debajo del boton "Empezar juego" para que el nombre no se encime con el.
838     int yLado = yArriba;
839     int altoLado = Math.Max(AnchoCarta, h - margen - yLado);
840     areasMano[1] = new Rectangle(w - margen - grosorMano, yLado, grosorMano, altoLado);
841     areasMano[3] = new Rectangle(margen, yLado, grosorMano, altoLado);
842 
843     for (int i = 0; i < NumJugadores; i++)
844         AjustarMano(i);
845 
846     // Centro: mazo a la izquierda y descarte a la derecha
847     int cx = w / 2;
848     int cy = (yArriba + grosorMano + yAbajo) / 2 - 20;
849     pictureBoxMazo.Location = new Point(cx - TamañoCentro.Width - 15, cy -
    TamañoCentro.Height / 2);
850     pictureBoxDescarte.Location = new Point(cx + 15, cy - TamañoCentro.Height / 2);
851     CentrarDebajo(label1, pictureBoxDescarte);
852     CentrarDebajo(botonPasar, pictureBoxMazo);
853     AcomodarBotonUno();
854 
855     ActualizarTurno();
856 }
```

AcomodarLayout (líneas 813 a 856) se ejecuta al cargar y al cambiar el tamaño de la ventana:

- Líneas 817 a 818: si la ventana está minimizada (tamaño 0) no hace nada.
- Línea 826: `grosorMano`: alto de carta + elevación + barra de scroll + 6.
- Líneas 829 a 834: las manos de arriba y abajo ocupan el ancho entre las columnas laterales.
- Líneas 838 a 841: las manos laterales ocupan casi toda la altura y empiezan a la misma altura que la mano de arriba (`yLado = yArriba`, línea 838). Antes empezaban debajo del botón "Empezar juego"; el comentario de la línea 837 todavía lo menciona aunque el botón ya no existe.
- Líneas 843 a 844: ajusta cada mano. Líneas 847 a 853 mazo a la izquierda del centro y descarte a la derecha; `label1` debajo del descarte, "Pasar" debajo del mazo y el botón UNO a la derecha.

Fragmento de Form1.cs (líneas 862 a 913)

```
862 private void AjustarMano(int posicion)
863 {
864     var area = areasMano[posicion];
865     var panel = manosJugadores[posicion];
866     bool lateral = EsLateral(posicion);
867 
868     int largo = panel.Controls.Cast<Control>()
869         .Sum(c => lateral ? c.Height + c.Margin.Vertical : c.Width + c.Margin.Horizontal);
870 
871     if (lateral)
872     {
873         int alto = Math.Min(largo, area.Height);
874         panel.Bounds = new Rectangle(area.X, area.Y + (area.Height - alto) / 2, area.Width,
    alto);
875     }
876     else
877     {
```

```
878         int ancho = Math.Min(largo, area.Width);
879         panel.Bounds = new Rectangle(area.X + (area.Width - ancho) / 2, area.Y, ancho,
    area.Height);
880     }
881 
882     var etiqueta = etiquetasNombre[posicion];
883     int centroX = panel.Left + panel.Width / 2;
884     // Se deja espacio a la derecha para el circulo de turno, sin salirse de la ventana
885     int espacioCirculo = indicadoresTurno[posicion].Width + 8;
886     int x = Math.Min(centroX - etiqueta.Width / 2, ClientSize.Width - etiqueta.Width -
    espacioCirculo);
887     etiqueta.Location = new Point(
888         Math.Max(0, x),
889         (lateral ? panel.Top : area.Top) - etiqueta.Height - 6);
890 }
891 
892 private static void CentrarDebajo(Control control, Control referencia)
893 {
894     control.Location = new Point(
895         referencia.Left + (referencia.Width - control.Width) / 2,
896         referencia.Bottom + 10);
897 }
898 
899 private static string Capitalizar(string texto)
900 {
901     return string.IsNullOrEmpty(texto) ? texto : char.ToUpper(texto[0]) +
    texto.Substring(1);
902 }
903 
904 /// <summary>
905 /// DoubleBuffered es protegido en los controles; se activa por reflexion
906 /// para que los paneles de cartas no parpadeen.
907 /// </summary>
908 private static void ActivarDobleBuffer(Control control)
909 {
910     typeof(Control)
911         .GetProperty("DoubleBuffered", System.Reflection.BindingFlags.NonPublic |
    System.Reflection.BindingFlags.Instance)
912         .SetValue(control, true, null);
913 }
```

- AjustarMano (líneas 862 a 890): calcula el largo real de las cartas (líneas 868 a 869, suma de tamaños + márgenes), hace el panel de ese tamaño (máximo su área) y lo centra en su área (líneas 871 a 880). Si no caben, el panel queda al máximo y aparece el scroll. Luego coloca el nombre encima, sin salirse de la ventana y dejando espacio para el ●.
- CentrarDebajo (líneas 892 a 897): coloca un control centrado 10 px debajo de otro.
- Capitalizar (líneas 899 a 902): primera letra mayúscula.
- ActivarDobleBuffer (líneas 908 a 913): `DoubleBuffered` es una propiedad protegida (solo accesible dentro de la propia clase del control). Con reflexión (`GetProperty` con `BindingFlags.NonPublic`) se activa desde fuera para que los paneles no parpadeen.

5.19 RobarCarta (líneas 915–933)

Fragmento de Form1.cs (líneas 915 a 933)

```
915 private void RobarCarta(int posicion)
916 {
917     // Sin partida en curso no hay jugadores (antes de empezar) o la partida ya termino.
918     // Tampoco se roba mientras alguien debe decir UNO.
919     if (!partidaEnCurso || jugadorEnUno >= 0)
920         return;
921 
922     if (DarCartas(posicion, 1) == 0)
923         return;
924 
925     // Puede seguir robando las veces que quiera, pero ya puede pasar
926     yaRobo = true;
927 
928     // Si la mano tiene scroll, se muestra la carta recien robada (queda al final)
929     var panel = manosJugadores[posicion];
930     panel.ScrollControlIntoView(panel.Controls[panel.Controls.Count - 1]);
931 
932     ActualizarTurno();
933 }
```

- Líneas 919 a 920: no se roba sin partida ni durante la cuenta de UNO.
- Líneas 922 a 923: da 1 carta; si no pudo (mazo vacío o error), sale.
- Línea 926: `yaRobo = true`: ya puede pasar. Se puede seguir robando sin límite (desviación de las reglas oficiales).
- Líneas 929 a 930: si la mano tiene scroll, la desplaza para que se vea la carta nueva (queda al final).

5.20 Decir UNO (líneas 943–1019)

Fragmento de Form1.cs (líneas 943 a 1019)

```
943 private void IniciarCuentaUno(int posicion)
944 {
945     jugadorEnUno = posicion;
946     segundosRestantesUno = SegundosParaUno;
947     ActualizarTextoUno();
948     botonUno.Visible = true;
949     ActualizarEtiqueta(posicion);
950     ActualizarTurno();
951     temporizadorUno.Start();
952 }
953 
954 private void AvanzarCuentaUno()
955 {
956     segundosRestantesUno--;
957     if (segundosRestantesUno > 0)
958     {
959         ActualizarTextoUno();
960         return;
961     }
962 
963     // Se acabo el tiempo: castigo de 2 cartas
964     int posicion = jugadorEnUno;
```

```
965     DetenerCuentaUno();
966     MessageBox.Show($"{jugadores[posicion].Nombre} no dijo UNO a tiempo. Roba 2 cartas de
    castigo.",
967         "UNO", MessageBoxButtons.OK, MessageBoxIcon.Warning);
968     DarCartas(posicion, 2, "penalizacion_uno");
969     ActualizarTurno();
970 }
971
972 /// <summary>
973 /// Clic en "¡UNO!" a tiempo: se registra en la base y no hay castigo.
974 /// </summary>
975 private void DecirUno()
976 {
977     if (jugadorEnUno < 0)
978         return;
979
980     int posicion = jugadorEnUno;
981     DetenerCuentaUno();
982     try
983     {
984         BaseDatos.RegistrarJugada(idPartida, jugadores[posicion].Id, "decir_uno", null);
985     }
986     catch (ErrorBaseDatos ex)
987     {
988         MostrarErrorBaseDatos(ex);
989     }
990
991     ActualizarEtiqueta(posicion); // ahora muestra "¡UNO!"
992     ActualizarTurno();
993 }
994
995 /// <summary>
996 /// Detiene la cuenta sin castigo (al decir UNO, empezar otra partida o cerrar).
997 /// </summary>
998 private void DetenerCuentaUno()
999 {
1000     temporizadorUno.Stop();
1001     jugadorEnUno = -1;
1002     botonUno.Visible = false;
1003 }
1004
1005 private void ActualizarTextoUno()
1006 {
1007     botonUno.Text = $"¡UNO! {jugadores[jugadorEnUno].Nombre} · {segundosRestantesUno}";
1008     AcomodarBotonUno();
1009 }
1010
1011 /// <summary>
1012 /// El boton va a la derecha de la carta del centro, a la misma altura.
1013 /// </summary>
1014 private void AcomodarBotonUno()
1015 {
1016     botonUno.Location = new Point(
1017         pictureBoxDescarte.Right + 25,
1018         pictureBoxDescarte.Top + (pictureBoxDescarte.Height - botonUno.Height) / 2);
1019 }
```

- IniciarCuentaUno (líneas 943 a 952): guarda quién debe decir UNO, pone 4 segundos, muestra el botón y arranca el timer. Desde este momento `jugadorEnUno >= 0` bloquea jugar, robar y pasar.

- AvanzarCuentaUno (líneas 954 a 970): se ejecuta cada segundo. Resta 1; si aún quedan segundos, actualiza el texto. Si llegó a 0: detiene la cuenta, avisa y da 2 cartas de castigo registradas como `penalizacion_uno`.
- DecirUno (líneas 975 a 993): clic a tiempo. Detiene la cuenta y registra `decir_uno` (sin carta). Si la API falla solo muestra el error (el jugador no se castiga). La etiqueta ahora muestra "¡UNO!".
- DetenerCuentaUno (líneas 998 a 1003): para el timer, `jugadorEnUno = -1` y oculta el botón. Se usa al decir UNO, al castigar, al empezar otra partida y al cerrar.
- ActualizarTextoUno (líneas 1005 a 1009): texto "¡UNO! Ana · 3" y reacomoda (el ancho cambia con el texto).
- AcomodarBotonUno (líneas 1014 a 1019): a la derecha del descarte, centrado verticalmente.

El juego queda "pausado" durante la cuenta sin detener nada: simplemente `JugarCarta`, `RobarCarta` y `botonPasar_Click` revisan `jugadorEnUno >= 0` al inicio y salen. Es la forma más simple de pausar un juego por turnos.

5.21 Reglas implementadas vs. reglas oficiales

Regla	**En este juego**	**UNO oficial**
Mazo	60 cartas, sin ceros	108 cartas
Carta inicial	Siempre de número	Puede ser especial (con efecto)
+4	Se puede jugar siempre	Solo si no tienes el color en juego
Acumular +2	No	Variante común (no oficial)
Robar	Ilimitado; después de robar se puede pasar	Robas 1 y puedes jugarla o pasar
Mazo agotado	La carta de arriba regresa al mazo en cada jugada	Se rebaraja el descarte
Decir UNO	Botón con 4 s; castigo de 2 cartas	Otro jugador debe "acusarte"

6. Las demás ventanas

6.1 FormInicio.cs: pantalla de inicio

Campos y constructor (líneas 13–66)

Fragmento de FormInicio.cs (líneas 13 a 66)

```
13  public partial class FormInicio : Form
14  {
15      // Colores del logo de UNO
16      private static readonly Color Amarillo = Color.FromArgb(255, 222, 0);
17      private static readonly Color Rojo = Color.FromArgb(220, 30, 30);
18
19      private readonly BotonUno botonJugar;
20      private readonly BotonUno botonSalir;
21      private readonly Label etiquetaEstado;
22      private bool apiDisponible = false;
23      private bool monitoreando = true;
24      // Hay un ciclo de revision de la API corriendo
25      private bool revisando;
26
27      public FormInicio()
28      {
29          InitializeComponent();
30
31          DoubleBuffered = true;
32          // Mismo icono que la ventana del juego
33          Icon = (Icon)new ComponentResourceManager(typeof(Ventana)).GetObject("$this.Icon");
34
35          botonJugar = new BotonUno
36          {
37              Text = "JUGAR",
38              Relleno = Amarillo,
39              ForeColor = Color.Black
40          };
41          botonJugar.Click += (s, e) => Jugar();
42
43          botonSalir = new BotonUno
44          {
45              Text = "SALIR",
46              Relleno = Rojo,
47              ForeColor = Color.White
48          };
49          botonSalir.Click += (s, e) => Close();
50
51          etiquetaEstado = new Label
52          {
53              AutoSize = true,
54              BackColor = Color.FromArgb(160, 0, 0, 0),
55              Padding = new Padding(10, 5, 10, 5),
56          };
57          // Si la API estaba apagada, un clic en el mensaje vuelve a revisar
58          //etiquetaEstado.Click += (s, e) => RevisarApi();
59
60          Controls.Add(botonJugar);
61          Controls.Add(botonSalir);
62          Controls.Add(etiquetaEstado);
63
```

```
64          Load += (s, e) => AcomodarVentana();
65          Shown += (s, e) => RevisarApi();
66      }
```

- Líneas 16 a 17: colores del logo de UNO (amarillo y rojo) con `FromArgb(r, g, b)`.
- Líneas 19 a 25: los dos botones, la etiqueta de estado, si la API está disponible, si se está monitoreando y `revisando` (evita dos ciclos a la vez).
- Línea 33: reutiliza el icono de la ventana del juego leyéndolo de sus recursos (`Form1.resx`).
- Líneas 35 a 49: crea los botones con la clase `BotonUno` (al final del archivo). JUGAR → `Jugar()`; SALIR → `Close()` (cierra el programa porque es la ventana principal).
- Líneas 51 a 56: etiqueta de estado con fondo semitransparente.
- Línea 58: está comentada la línea que reintentaba con un clic (ya no hace falta porque el ciclo revisa solo cada 2 s).
- Línea 64: al cargar se acomoda la ventana; línea 65 cuando ya se ve, empieza a revisar la API.

AcomodarVentana y AcomodarEstado (líneas 73–108)

Fragmento de FormInicio.cs (líneas 73 a 108)

```
73  private void AcomodarVentana()
74  {
75      var pantalla = Screen.FromControl(this).WorkingArea;
76      int ancho = Math.Min(1280, pantalla.Width * 85 / 100);
77      int alto = ancho * 9 / 16;
78      if (alto > pantalla.Height * 85 / 100)
79      {
80          alto = pantalla.Height * 85 / 100;
81          ancho = alto * 16 / 9;
82      }
83      ClientSize = new Size(ancho, alto);
84      CenterToScreen();
85
86      // Botones debajo del logo (el logo llega hasta ~74% de la altura de la imagen)
87      int altoBoton = alto * 11 / 100;
88      botonJugar.Size = new Size(ancho * 26 / 100, altoBoton);
89      botonSalir.Size = new Size(ancho * 14 / 100, altoBoton);
90      botonJugar.Font = new Font("Segoe UI", altoBoton * 0.42f, FontStyle.Bold,
    GraphicsUnit.Pixel);
91      botonSalir.Font = new Font("Segoe UI", altoBoton * 0.36f, FontStyle.Bold,
    GraphicsUnit.Pixel);
92
93      int separacion = ancho * 2 / 100;
94      int y = alto * 79 / 100;
95      int x = (ancho - botonJugar.Width - separacion - botonSalir.Width) / 2;
96      botonJugar.Location = new Point(x, y);
97      botonSalir.Location = new Point(botonJugar.Right + separacion, y);
98
99      etiquetaEstado.Font = new Font("Segoe UI", Math.Max(9f, alto * 0.018f), FontStyle.Bold,
    GraphicsUnit.Point);
100     AcomodarEstado();
101 }
102
```

```
103 private void AcomodarEstado()
104 {
105     etiquetaEstado.Location = new Point(
106         (ClientSize.Width - etiquetaEstado.Width) / 2,
107         ClientSize.Height - etiquetaEstado.Height - ClientSize.Height * 2 / 100);
108 }
```

- Línea 75: el área útil de la pantalla (sin la barra de tareas).
- Líneas 76 a 82: ancho = 85 % de la pantalla (máx. 1280), alto en proporción 16:9 (`ancho * 9 / 16`); si no cabe en alto, se calcula al revés. Así la imagen de fondo no se deforma.
- Líneas 87 a 97: los botones se calculan como porcentajes del tamaño de la ventana (11 % de alto, 26 % y 14 % de ancho, a 79 % de altura, debajo del logo). Las fuentes también son proporcionales.
- Líneas 103 a 108: la etiqueta de estado va centrada abajo.

Revisión de la API (líneas 113–157)

Fragmento de FormInicio.cs (líneas 113 a 157)

```
113 private async void FormInicio_Load(object sender, EventArgs e)
114 {
115     await RevisarApi();
116 }
117 private async Task RevisarApi()
118 {
119     // Si ya hay un ciclo corriendo (por ejemplo, al volver de la mesa en menos de
120     // 2 segundos el ciclo anterior sigue vivo), no se inicia otro
121     if (revisando)
122         return;
123     revisando = true;
124     try
125     {
126         await CicloRevisarApi();
127     }
128     finally
129     {
130         revisando = false;
131     }
132 }
133
134 private async Task CicloRevisarApi()
135 {
136     while (monitoreando)
137     {
138         apiDisponible = await Task.Run(() => BaseDatos.ApiDisponible());
139
140         if (apiDisponible)
141         {
142             etiquetaEstado.Text = "● Conectado";
143             etiquetaEstado.ForeColor = Color.LightGreen;
144         }
145         else
146         {
147             etiquetaEstado.Text = "● Error de conexión";
148             etiquetaEstado.ForeColor = Color.FromArgb(255, 160, 150);
149         }
150
```

```
151         botonJugar.Enabled = apiDisponible;
152         AcomodarEstado();
153
154         // Espera 3 segundos ANTES de volver a iniciar el ciclo
155         await Task.Delay(2000);
156     }
157 }
```

- Líneas 117 a 132 RevisarApi: si ya hay un ciclo corriendo no inicia otro (líneas 121 a 122). `try/finally` asegura que `revisando` vuelva a `false`.
- Líneas 134 a 157 CicloRevisarApi: mientras `monitoreando` sea `true`: llama a `ApiDisponible` en otro hilo con `Task.Run` (línea 138) para no congelar la ventana; muestra "● Conectado" en verde o "● Error de conexión" en rojo claro; habilita JUGAR solo si hay conexión (línea 151); espera 2 segundos (línea 155) y repite.

Observación: El comentario de línea 154 dice "3 segundos" pero el código espera 2000 ms (2 s). Además, `FormInicio_Load` (líneas 113 a 116) no está conectado a ningún evento (ni en el constructor ni en el diseñador), así que nunca se ejecuta; la revisión se inicia en línea 65.

Jugar (líneas 163–183)

Fragmento de FormInicio.cs (líneas 163 a 183)

```
163 private async Task Jugar()
164 {
165     if (!apiDisponible)
166         return;
167     monitoreando = false;
168     Hide();
169
170     using (var juego = new Ventana())
171     {
172         // La partida empieza cuando la mesa ya se ve y esta acomodada
173         juego.Shown += (s, e) =>
174         {
175             if (!juego.EmpezarPartida())
176                 juego.Close();
177         };
178         juego.ShowDialog();
179     }
180     Show();
181     monitoreando = true;
182     await RevisarApi();
183 }
```

- Líneas 165 a 166: sin API no se puede jugar.
- Líneas 167 a 168: detiene el ciclo de revisión y oculta el inicio.
- Líneas 170 a 179: crea la mesa. Cuando ya está visible (`Shown`), llama a `EmpezarPartida()`; si regresa `false` (se canceló la ventana de nombres) cierra la mesa. `ShowDialog()` espera aquí hasta que la mesa se cierre.
- Líneas 180 a 182: vuelve a mostrar el inicio y reanuda el monitoreo.

Clase BotonUno (líneas 191–259)

Fragmento de FormInicio.cs (líneas 191 a 259)

```
191 internal class BotonUno : Control
192 {
193     private bool encima;
194     private bool presionado;
195 
196     public Color Relleno { get; set; } = Color.Gold;
197 
198     public BotonUno()
199     {
200         SetStyle(ControlStyles.UserPaint | ControlStyles.AllPaintingInWmPaint |
201                  ControlStyles.OptimizedDoubleBuffer | ControlStyles.ResizeRedraw |
202                  ControlStyles.SupportsTransparentBackColor, true);
203         BackColor = Color.Transparent;
204         Cursor = Cursors.Hand;
205     }
206 
207     protected override void OnMouseEnter(EventArgs e) { encima = true; Invalidate();
    base.OnMouseEnter(e); }
208     protected override void OnMouseLeave(EventArgs e) { encima = false; presionado = false;
    Invalidate(); base.OnMouseLeave(e); }
209     protected override void OnMouseDown(MouseEventArgs e) { presionado = true; Invalidate();
    base.OnMouseDown(e); }
210     protected override void OnMouseUp(MouseEventArgs e) { presionado = false; Invalidate();
    base.OnMouseUp(e); }
211     protected override void OnEnabledChanged(EventArgs e) { Cursor = Enabled ? Cursors.Hand
    : Cursors.Default; Invalidate(); base.OnEnabledChanged(e); }
212 
213     protected override void OnPaint(PaintEventArgs e)
214     {
215         var g = e.Graphics;
216         g.SmoothingMode = SmoothingMode.AntiAlias;
217         g.TextRenderingHint = System.Drawing.Text.TextRenderingHint.AntiAliasGridFit;
218 
219         int grosor = Math.Max(3, Height / 14);
220         int sombra = Math.Max(3, Height / 12);
221         // Al presionar, el boton "baja" y tapa su sombra
222         int baja = presionado ? sombra : 0;
223         var rect = new Rectangle(grosor / 2, grosor / 2 + baja,
224                                  Width - grosor - 1, Height - grosor - sombra - 1);
225 
226         Color fondo = !Enabled ? Color.FromArgb(150, 150, 150)
227                     : encima ? ControlPaint.Light(Relleno, 0.4f)
228                     : Relleno;
229 
230         using (var forma = Pildora(rect))
231         {
232             if (!presionado)
233             {
234                 using (var formaSombra = Pildora(new Rectangle(rect.X, rect.Y + sombra,
    rect.Width, rect.Height)))
235                 using (var pincelSombra = new SolidBrush(Color.FromArgb(140, 0, 0, 0)))
236                     g.FillPath(pincelSombra, formaSombra);
237             }
238             using (var pincel = new SolidBrush(fondo))
239                 g.FillPath(pincel, forma);
240             using (var pluma = new Pen(Color.Black, grosor))
241                 g.DrawPath(pluma, forma);
242         }
243 
```

```
244         TextRenderer.DrawText(g, Text, Font, rect, Enabled ? ForeColor : Color.FromArgb(70,
    70, 70),
245             TextFormatFlags.HorizontalCenter | TextFormatFlags.VerticalCenter |
    TextFormatFlags.SingleLine);
246     }
247
248     private static GraphicsPath Pildora(Rectangle r)
249     {
250         int d = Math.Min(r.Height, r.Width);
251         var path = new GraphicsPath();
252         path.AddArc(r.X, r.Y, d, d, 90, 180);
253         path.AddArc(r.Right - d, r.Y, d, d, 270, 180);
254         path.CloseFigure();
255         return path;
256     }
257
258
259 }
```

Un botón dibujado a mano que hereda de `Control` (la clase base de todos los controles).

- Líneas 193 a 196: estado: mouse encima, presionado, y el color de relleno.
- Líneas 200 a 202: `SetStyle`: el control se dibuja solo (`UserPaint`), con doble buffer, se redibuja al cambiar de tamaño y acepta fondo transparente.
- Líneas 207 a 211: al entrar/salir el mouse, presionar o soltar, cambia el estado y llama a `Invalidate()` (pide redibujar). Si se deshabilita, el cursor deja de ser una mano.
- Líneas 213 a 246 OnPaint: suavizado (líneas 216 a 217). Grosor del borde y sombra proporcionales al alto (líneas 219 a 220). Al presionar, el botón "baja" (línea 222). Color: gris si está deshabilitado, más claro si el mouse está encima (`ControlPaint.Light`), o el normal (líneas 226 a 228). Dibuja la sombra (solo si no está presionado), el relleno y el borde negro, y el texto centrado.
- Líneas 248 a 256 Pildora: crea la forma de píldora con dos medios círculos (`AddArc` de 180°) unidos con `CloseFigure()`.

6.2 FormNombres.cs: capturar los nombres

Fragmento de FormNombres.cs (líneas 12 a 99)

```
12 public class FormNombres : Form
13 {
14     private const int LargoMaximo = 30; // igual que nombre_jug varchar(30)
15
16     private readonly TextBox[] cajasNombre;
17
18     /// <summary>
19     /// Nombres capturados, en orden de posicion. Solo es valido si DialogResult == OK.
20     /// </summary>
21     public List<string> Nombres { get; private set; } = new List<string>();
22
23     public FormNombres(int cantidad, IList<string> nombresSugeridos)
24     {
25         Text = "Nombres de los jugadores";
26         FormBorderStyle = FormBorderStyle.FixedDialog;
```

```
        StartPosition = FormStartPosition.CenterParent;
        MaximizeBox = false;
        MinimizeBox = false;
        AutoSize = true;
        AutoSizeMode = AutoSizeMode.GrowAndShrink;
        Padding = new Padding(12);

        var tabla = new TableLayoutPanel { ColumnCount = 2, AutoSize = true, Dock =
DockStyle.Fill };
        cajasNombre = new TextBox[cantidad];

        for (int i = 0; i < cantidad; i++)
        {
            tabla.Controls.Add(new Label
            {
                Text = $"Jugador {i + 1}:",
                AutoSize = true,
                Anchor = AnchorStyles.Left,
                Margin = new Padding(3, 8, 3, 3)
            }, 0, i);

            cajasNombre[i] = new TextBox
            {
                Width = 200,
                MaxLength = LargoMaximo,
                Text = i < nombresSugeridos.Count ? nombresSugeridos[i] : ""
            };
            tabla.Controls.Add(cajasNombre[i], 1, i);
        }

        var botonAceptar = new Button { Text = "Aceptar", AutoSize = true };
        var botonCancelar = new Button { Text = "Cancelar", AutoSize = true, DialogResult =
DialogResult.Cancel };
        botonAceptar.Click += BotonAceptar_Click;

        var botones = new FlowLayoutPanel
        {
            FlowDirection = FlowDirection.RightToLeft,
            AutoSize = true,
            Dock = DockStyle.Fill,
            Margin = new Padding(0, 10, 0, 0)
        };
        botones.Controls.Add(botonCancelar);
        botones.Controls.Add(botonAceptar);
        tabla.Controls.Add(botones, 0, cantidad);
        tabla.SetColumnSpan(botones, 2);

        Controls.Add(tabla);
        AcceptButton = botonAceptar;
        CancelButton = botonCancelar;
    }

    private void BotonAceptar_Click(object sender, EventArgs e)
    {
        var nombres = cajasNombre.Select(c => c.Text.Trim()).ToList();

        int vacio = nombres.FindIndex(string.IsNullOrEmpty);
        if (vacio >= 0)
        {
            MessageBox.Show($"Escribe el nombre del jugador {vacio + 1}.", "UNO",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            cajasNombre[vacio].Focus();
```

```
87              return;
88          }
89
90          if (nombres.Distinct(StringComparer.OrdinalIgnoreCase).Count() < nombres.Count)
91          {
92              MessageBox.Show("Cada jugador debe tener un nombre distinto.", "UNO",
93                  MessageBoxButtons.OK, MessageBoxIcon.Warning);
94              return;
95          }
96
97          Nombres = nombres;
98          DialogResult = DialogResult.OK;
99      }
```

Esta ventana se crea solo con código (sin diseñador).

- Línea 14: máximo 30 caracteres, igual que la columna `nombre_jug`.
- Línea 21: `Nombres` es público para leer (`get`) pero solo se asigna adentro (`private set`).
- Líneas 26 a 32: ventana de diálogo fija, centrada sobre la mesa, sin maximizar, que se ajusta a su contenido.
- Línea 34: un `TableLayoutPanel` de 2 columnas (etiqueta | caja de texto).
- Líneas 37 a 54: una fila por jugador. La caja trae el nombre sugerido si existe (línea 51).
- Líneas 56 a 70: botones Aceptar y Cancelar en una fila que ocupa las 2 columnas. Cancelar tiene `DialogResult.Cancel`: al presionarlo la ventana se cierra sola con ese resultado.
- Líneas 73 a 74: Enter = Aceptar, Esc = Cancelar.
- Líneas 77 a 99 BotonAceptar_Click: quita espacios (`Trim`); si algún nombre está vacío avisa y pone el cursor en esa caja (líneas 81 a 88); si hay nombres repetidos sin importar mayúsculas (`StringComparer.OrdinalIgnoreCase`) avisa (líneas 90 a 95). Si todo está bien guarda los nombres y pone `DialogResult = OK`, lo que cierra la ventana.

Nota: La validación de repetidos sin distinguir mayúsculas coincide con MySQL: con la configuración normal, MySQL considera "ana" y "Ana" iguales, así que la llave `unique` también los trataría como el mismo jugador.

Observación: `InitializeComponent` (líneas 101 a 111) fue generado por Visual Studio y nunca se llama (el constructor no lo usa). No afecta el funcionamiento.

6.3 FormElegirColor: elegir color del comodín

Fragmento de FormElegirColor.cs (líneas 13 a 56)

```
13  public partial class FormElegirColor : Form
14  {
15      private string colorSeleccionado;
16      public FormElegirColor()
17      {
18          InitializeComponent();
19
20          // La ventana se ajusta a las 4 cartas; con el tamaño fijo del diseñador
```

```
21          // la amarilla quedaba cortada en pantallas con otra escala (DPI)
22          AutoSize = true;
23          AutoSizeMode = AutoSizeMode.GrowAndShrink;
24      }
25      public string getColor()
26      {
27          return this.colorSeleccionado;
28      }
29      private void pictureBox3_Click(object sender, EventArgs e)
30      {
31          colorSeleccionado = "verde";
32          this.DialogResult = DialogResult.OK;
33          this.Close();
34      }
35
36      private void pictureBox4_Click(object sender, EventArgs e)
37      {
38          colorSeleccionado = "amarillo";
39          this.DialogResult = DialogResult.OK;
40          this.Close();
41      }
42
43      private void pictureBox2_Click(object sender, EventArgs e)
44      {
45          colorSeleccionado = "azul";
46          this.DialogResult = DialogResult.OK;
47          this.Close();
48      }
49
50      private void pictureBox1_Click(object sender, EventArgs e)
51      {
52          colorSeleccionado = "rojo";
53          this.DialogResult = DialogResult.OK;
54          this.Close();
55      }
56  }
```

- Líneas 22 a 23: la ventana se ajusta a las 4 imágenes (antes la amarilla se cortaba en pantallas con otra escala de DPI).
- Líneas 25 a 28: `getColor()` devuelve el color elegido.
- Líneas 29 a 55: un manejador por imagen. Al hacer clic guarda el color, pone `DialogResult.OK` y cierra.

Control	**Posición en la ventana**	**Color**
`pictureBox1`	1ª (x = 7)	rojo
`pictureBox2`	2ª (x = 260)	azul
`pictureBox3`	3ª (x = 513)	verde
`pictureBox4`	4ª (x = 767)	amarillo

En `FormElegirColor.Designer.cs` cada `PictureBox` tiene cursor de mano, imagen desde el `.resx`, tamaño 247×332 y su evento `Click` conectado (ej. Línea 53). Línea 99 `ControlBox = false` quita los botones de la barra de título (no se puede cerrar con la X, hay que elegir un color).

6.4 FormGanador.cs: pantalla del ganador

Fragmento de FormGanador.cs (líneas 13 a 78)

```
13  public class FormGanador : Form
14  {
15      private static readonly Color Amarillo = Color.FromArgb(255, 222, 0);
16      private static readonly Color[] ColoresUno =
17      {
18          Color.FromArgb(220, 30, 30), Color.FromArgb(255, 222, 0),
19          Color.FromArgb(30, 110, 220), Color.FromArgb(40, 170, 70), Color.White
20      };
21
22      private readonly string nombreGanador;
23      private readonly int idPartida;
24      private readonly List<Confeti> confeti = new List<Confeti>();
25      private readonly Random aleatorio = new Random();
26      private readonly Timer animacion = new Timer { Interval = 30 };
27      private readonly BotonUno botonVolver;
28
29      private class Confeti
30      {
31          public float X, Y, VelocidadY, VelocidadX, Angulo, Giro;
32          public int Ancho, Alto;
33          public Color Color;
34      }
35
36      public FormGanador(string nombreGanador, int idPartida)
37      {
38          this.nombreGanador = nombreGanador;
39          this.idPartida = idPartida;
40
41          Text = "¡Tenemos ganador!";
42          FormBorderStyle = FormBorderStyle.None;
43          StartPosition = FormStartPosition.CenterParent;
44          ShowInTaskbar = false;
45          ClientSize = new Size(640, 420);
46          DoubleBuffered = true;
47          KeyPreview = true;
48
49          botonVolver = new BotonUno
50          {
51              Text = "VOLVER AL INICIO",
52              Relleno = Amarillo,
53              ForeColor = Color.Black,
54              Size = new Size(300, 64),
55              Font = new Font("Segoe UI", 20, FontStyle.Bold, GraphicsUnit.Pixel)
56          };
57          botonVolver.Location = new Point((ClientSize.Width - botonVolver.Width) / 2,
    ClientSize.Height - botonVolver.Height - 30);
58          botonVolver.Click += (s, e) => Close();
59          Controls.Add(botonVolver);
60
61          // Enter o Esc tambien cierran, como un "Aceptar"
62          KeyDown += (s, e) =>
63          {
64              if (e.KeyCode == Keys.Enter || e.KeyCode == Keys.Escape)
65                  Close();
66          };
67
68          for (int i = 0; i < 70; i++)
69              confeti.Add(NuevoConfeti(aleatorio.Next(ClientSize.Height)));
```

```
70
71          animacion.Tick += (s, e) => MoverConfeti();
72          Shown += (s, e) =>
73          {
74              System.Media.SystemSounds.Asterisk.Play();
75              animacion.Start();
76          };
77          FormClosed += (s, e) => animacion.Stop();
78      }
```

- Líneas 16 a 20: los colores del confeti (rojo, amarillo, azul, verde, blanco).
- Líneas 24 a 27: la lista de confeti, un `Timer` de 30 ms (≈ 33 cuadros por segundo) y el botón.
- Líneas 29 a 34: clase interna `Confeti`: posición, velocidades, ángulo, giro, tamaño y color de cada papelito.
- Líneas 42 a 47: ventana sin bordes, centrada, de 640×420, con doble buffer y `KeyPreview` (la ventana recibe las teclas antes que el botón).
- Líneas 49 a 59: botón "VOLVER AL INICIO" centrado abajo; cierra la ventana.
- Líneas 62 a 66: Enter o Esc también cierran.
- Líneas 68 a 69: crea 70 papelitos en alturas aleatorias.
- Líneas 71 a 77: cada tic mueve el confeti; al mostrarse suena un sonido del sistema y arranca la animación; al cerrar la detiene.

Fragmento de FormGanador.cs (líneas 80 a 108)

```
80  private Confeti NuevoConfeti(float y)
81  {
82      return new Confeti
83      {
84          X = aleatorio.Next(ClientSize.Width),
85          Y = y,
86          VelocidadY = 1.5f + (float)aleatorio.NextDouble() * 2.5f,
87          VelocidadX = (float)aleatorio.NextDouble() * 1.2f - 0.6f,
88          Angulo = aleatorio.Next(360),
89          Giro = (float)aleatorio.NextDouble() * 8 - 4,
90          Ancho = aleatorio.Next(6, 12),
91          Alto = aleatorio.Next(10, 18),
92          Color = ColoresUno[aleatorio.Next(ColoresUno.Length)]
93      };
94  }
95
96  private void MoverConfeti()
97  {
98      for (int i = 0; i < confeti.Count; i++)
99      {
100         var c = confeti[i];
101         c.Y += c.VelocidadY;
102         c.X += c.VelocidadX;
103         c.Angulo += c.Giro;
104         if (c.Y > ClientSize.Height + 20)
105             confeti[i] = NuevoConfeti(-20); // vuelve a caer desde arriba
106     }
107     Invalidate();
108 }
```

- NuevoConfeti: valores aleatorios: cae a 1.5–4 px por cuadro, se mueve de lado entre −0.6 y 0.6, gira entre −4° y 4° por cuadro.
- MoverConfeti: suma la velocidad a la posición y el giro al ángulo. Si un papelito sale por abajo, se reemplaza por uno nuevo arriba (línea 105). `Invalidate()` pide redibujar.

Fragmento de FormGanador.cs (líneas 110 a 189)

```
110 protected override void OnPaint(PaintEventArgs e)
111 {
112     var g = e.Graphics;
113     g.SmoothingMode = SmoothingMode.AntiAlias;
114     var area = ClientRectangle;
115 
116     // Fondo: degradado rojo, como la mesa
117     using (var fondo = new LinearGradientBrush(area, Color.FromArgb(235, 45, 25),
    Color.FromArgb(120, 0, 0), 90f))
118         g.FillRectangle(fondo, area);
119 
120     // Ovalo rojo inclinado con borde amarillo, como el logo de UNO
121     var estado = g.Save();
122     g.TranslateTransform(area.Width / 2f, 150);
123     g.RotateTransform(-8);
124     var ovalo = new RectangleF(-260, -105, 520, 210);
125     using (var sombra = new SolidBrush(Color.FromArgb(90, 0, 0, 0)))
126         g.FillEllipse(sombra, ovalo.X + 8, ovalo.Y + 10, ovalo.Width, ovalo.Height);
127     using (var rojo = new SolidBrush(Color.FromArgb(220, 20, 20)))
128         g.FillEllipse(rojo, ovalo);
129     using (var borde = new Pen(Amarillo, 8))
130         g.DrawEllipse(borde, ovalo);
131     g.Restore(estado);
132 
133     // Confeti
134     foreach (var c in confeti)
135     {
136         var est = g.Save();
137         g.TranslateTransform(c.X, c.Y);
138         g.RotateTransform(c.Angulo);
139         using (var pincel = new SolidBrush(c.Color))
140             g.FillRectangle(pincel, -c.Ancho / 2f, -c.Alto / 2f, c.Ancho, c.Alto);
141         g.Restore(est);
142     }
143 
144     // Textos con contorno negro, al estilo del logo
145     TextoConContorno(g, "¡GANADOR!", 64, Amarillo, new RectangleF(0, 65, area.Width, 90));
146     TextoConContorno(g, nombreGanador, AjustarTamaño(g, nombreGanador, 46, area.Width - 80),
147         Color.White, new RectangleF(0, 150, area.Width, 70));
148     TextoConContorno(g, $"ganó la partida #{idPartida}", 20, Color.White, new RectangleF(0,
    250, area.Width, 40));
149 
150     // Marco amarillo y negro
151     using (var negro = new Pen(Color.Black, 10))
152         g.DrawRectangle(negro, 5, 5, area.Width - 10, area.Height - 10);
153     using (var amarillo = new Pen(Amarillo, 4))
154         g.DrawRectangle(amarillo, 12, 12, area.Width - 24, area.Height - 24);
155 }
156 
157 /// <summary>
158 /// Achica la letra si el nombre es muy largo para que no se salga del recuadro.
159 /// </summary>
```

```
160 private static float AjustarTamaño(Graphics g, string texto, float tamaño, float
    anchoMaximo)
161 {
162     using (var familia = new FontFamily("Segoe UI"))
163     {
164         while (tamaño > 16)
165         {
166             using (var fuente = new Font(familia, tamaño, FontStyle.Bold,
    GraphicsUnit.Pixel))
167             {
168                 if (g.MeasureString(texto, fuente).Width <= anchoMaximo)
169                     break;
170             }
171             tamaño -= 2;
172         }
173     }
174     return tamaño;
175 }
176
177 private static void TextoConContorno(Graphics g, string texto, float tamaño, Color relleno,
    RectangleF area)
178 {
179     using (var familia = new FontFamily("Segoe UI"))
180     using (var formato = new StringFormat { Alignment = StringAlignment.Center,
    LineAlignment = StringAlignment.Center })
181     using (var trazo = new GraphicsPath())
182     {
183         trazo.AddString(texto, familia, (int)FontStyle.Bold, tamaño, area, formato);
184         using (var contorno = new Pen(Color.Black, Math.Max(3f, tamaño / 7f)) { LineJoin =
    LineJoin.Round })
185             g.DrawPath(contorno, trazo);
186         using (var pincel = new SolidBrush(relleno))
187             g.FillPath(pincel, trazo);
188     }
189 }
```

- Líneas 117 a 118: fondo con degradado rojo de arriba hacia abajo.
- Líneas 121 a 131: óvalo rojo inclinado −8° con borde amarillo, como el logo. `Save`/`TranslateTransform`/`RotateTransform`/`Restore`: mueve y gira el "lápiz", dibuja y vuelve a la normalidad.
- Líneas 134 a 142: cada papelito se dibuja trasladado a su posición y girado.
- Líneas 145 a 148: textos "¡GANADOR!", el nombre y "ganó la partida #N".
- Líneas 151 a 154: marco negro y amarillo.
- AjustarTamaño (líneas 160 a 175): reduce la letra de 2 en 2 hasta que el nombre quepa (mínimo 16).
- TextoConContorno (líneas 177 a 189): convierte el texto en una figura (`GraphicsPath.AddString`), dibuja el contorno negro grueso y lo rellena del color.

7. Archivos del diseñador y de configuración

7.1 Form1.Designer.cs

Lo genera Visual Studio al arrastrar controles. Es la otra mitad de la clase `partial Ventana`.

Fragmento de Form1.Designer.cs (líneas 15 a 22)

```
15  protected override void Dispose(bool disposing)
16  {
17      if (disposing && (components != null))
18      {
19          components.Dispose();
20      }
21      base.Dispose(disposing);
22  }
```

Dispose (líneas 15 a 22): libera los componentes al cerrar la ventana.

Control	Líneas	Uso en el juego
`pictureBox4`	33 y 49–58	Descarte (carta de arriba). Se renombra `pictureBoxDescarte` en Form1.cs línea 79.
`pictureBox6`	40 y 132–141	Mazo (imagen de reverso). Se renombra `pictureBoxMazo` en línea 80.
`lblJugador1..4`	34–37 y 60–106	Nombres de los jugadores.
`botonPasar`	38 y 111–120	"Pasar". `Click` → `botonPasar_Click`.
`label1`	39 y 122–130	"Color en juego".
`flowMano1..4`	41–44 y 143–183	Paneles con las cartas de cada jugador.

Las líneas 108 y 109 son el comentario `// button9` que dejó el diseñador vacío cuando se borró el botón "Empezar juego".

Fragmento de Form1.Designer.cs (líneas 185 a 215)

```
185 // Ventana
186 //
187 this.AutoScaleDimensions = new System.Drawing.SizeF(12F, 25F);
188 this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
189 this.BackColor = System.Drawing.Color.Red;
190 this.BackgroundImage =
    ((System.Drawing.Image)(resources.GetObject("$this.BackgroundImage")));
191 this.BackgroundImageLayout = System.Windows.Forms.ImageLayout.Stretch;
192 this.ClientSize = new System.Drawing.Size(2292, 1258);
193 this.Controls.Add(this.flowMano2);
194 this.Controls.Add(this.flowMano3);
195 this.Controls.Add(this.flowMano4);
196 this.Controls.Add(this.flowMano1);
197 this.Controls.Add(this.pictureBox6);
198 this.Controls.Add(this.botonPasar);
199 this.Controls.Add(this.label1);
200 this.Controls.Add(this.lblJugador4);
201 this.Controls.Add(this.lblJugador3);
```

```
202 this.Controls.Add(this.lblJugador2);
203 this.Controls.Add(this.lblJugador1);
204 this.Controls.Add(this.pictureBox4);
205 this.FormBorderStyle = System.Windows.Forms.FormBorderStyle.FixedSingle;
206 this.Icon = ((System.Drawing.Icon)(resources.GetObject("$this.Icon")));
207 this.Margin = new System.Windows.Forms.Padding(4, 3, 4, 3);
208 this.MaximizeBox = false;
209 this.Name = "Ventana";
210 this.Text = "UNO";
211 this.WindowState = System.Windows.Forms.FormWindowState.Maximized;
212 ((System.ComponentModel.ISupportInitialize)(this.pictureBox4)).EndInit();
213 ((System.ComponentModel.ISupportInitialize)(this.pictureBox6)).EndInit();
214 this.ResumeLayout(false);
215 this.PerformLayout();
```

- Líneas 189 a 191: fondo rojo con imagen de fondo estirada.
- Línea 205: borde fijo; línea 208 sin maximizar; línea 211 empieza maximizada (pantalla completa).
- Línea 206: icono desde `Form1.resx`.

Nota: Las posiciones y tamaños que pone el diseñador (por ejemplo la línea 52) no se usan realmente: `AcomodarLayout` y `AplicarEstilo` en Form1.cs los recalculan. El diseñador solo "crea" los controles.

7.2 FormInicio.Designer.cs

Solo configura la ventana: imagen de fondo desde `FormInicio.resx` estirada (líneas 38 a 39), borde fijo, sin maximizar, centrada, título "UNO". Los botones se crean en código.

7.3 Configuración del proyecto

Archivo	Qué contiene
`Juego-UNO.csproj`	Proyecto de tipo `WinExe` (aplicación de ventanas, línea 8) para .NET Framework 4.7.2 (línea 11). Lista de archivos a compilar, referencias, y las 50 imágenes de `Cartas` con `CopyToOutputDirectory = PreserveNewest` (se copian junto al .exe; por eso `CargarImagen` las busca en `Application.StartupPath\Cartas`).
`packages.config`	Paquetes NuGet: Newtonsoft.Json 13.0.3 (el que se usa para JSON) y MySqlConnector 2.6.2 con sus dependencias (de cuando C# se conectaba a MySQL directo; ya no se usa).
`App.config`	Versión de .NET y binding redirects (le dicen a .NET qué versión de una librería usar si dos paquetes piden versiones distintas).
`Properties/AssemblyInfo.cs`	Nombre, versión 1.0.0.0 y GUID del programa.
`Properties/Resources.*`, `Settings.*`	Generados automáticamente por Visual Studio; el juego no los usa.

Archivo	Qué contiene
`*.resx`	Recursos incrustados en el .exe: icono, fondos, imágenes de colores (por eso `FormElegirColor.resx` mide 62 mil líneas: son imágenes en base64).
`.gitignore`	Ignora `bin/`, `obj/`, `.vs/`, `.venv/`, `.env`, `__pycache__/`, etc.
`.env.ejemplo`	Plantilla: `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME=unobd`, `DB_PORT=3306`.
`librerias.txt`	`fastapi`, `uvicorn` (servidor), `pydantic` (validación), `mysql-connector-python`, `python-dotenv` y sus dependencias.

8. Historial de commits: qué hizo cada quien

8.1 Resumen por integrante

El repositorio tiene 77 commits entre el 21 de septiembre y el 10 de octubre de 2026, en 6 ramas y con 20 pull requests. Algunas personas aparecen con dos nombres porque hicieron commits desde su computadora y desde la página de GitHub (los merge de pull requests):

Integrante	Nombres en Git	Rama	Commits
Victor	`Victor`, `Víctor G`	`victor`, `victor2.0`	36 (14 son merges)
Joaquin	`Joaquin`, `Joaquin Z`	`rama_joaquin`	17 (1 es merge)
Ximena	`Ximena Ovalle`, `Ximena814`	`xop`	17 (6 son merges)
Angel	`Angel`, `Angel-20-TOO`	`rama_angel`	7 (1 es merge)

Integrante	Aportaciones principales
Joaquin	Creó el proyecto en Visual Studio, `.gitignore`, icono y nombre de la ventana. Primer paquete de MySQL y el jugador 4 en la interfaz. Lógica del +2, validación de cartas, validación del +4 y cambio de color, y la ventana para elegir color. Creó la primera API (`db.py`, `main.py` inicial, `librerias.txt`). Al final: etiqueta de inicio, cursor en elegir color, quitar botones sin uso y el efecto rojo para cartas inválidas.
Ximena	Primer script SQL de la base de datos. Imagen de fondo de la mesa. Corrección de robar carta. Imágenes de la ventana de elegir color. Primera pantalla de inicio y conectar su botón con el método de empezar partida.
Angel	Imágenes reales de las cartas (las 50 PNG). Ventana en pantalla completa con tamaño fijo. Quitó los botones/ComboBox antiguos de jugar y robar y creó los paneles transparentes donde las cartas se muestran como imágenes, con doble clic para jugar y scroll. Al final borró `CONTEXTO_PROYECTO_UNO.md` del repositorio.
Victor	Primera interfaz, clases `Carta`/`Jugador`, conexión y reparto. Robar y jugar, turnos, reversa, +2, +4. Validar antes de registrar, color elegido de comodines, `colorActual`, mismo símbolo, botón Pasar. API completa y adaptación de `BaseDatos.cs` a HTTP/JSON. Arreglo del conflicto de merge en el diseñador. Refactorización de la interfaz y de la pantalla de inicio. Botón ¡UNO!, pantalla del ganador y documento de contexto. Aprobó la mayoría de los pull requests.

Líneas de cada integrante que siguen en la versión final (`git blame`, rama master, commit `9870aa0`):

Archivo	Victor	Joaquin	Ximena	Angel
`Form1.cs`	862	85	19	55
`Form1.Designer.cs`	78	55	53	48
`FormInicio.cs`	214	28	18	–

Archivo	Victor	Joaquin	Ximena	Angel
`FormElegirColor.cs` + Designer	5	120	56	–
`FormNombres.cs`	101	–	12	–
`BaseDatos.cs`, `FormGanador.cs`, `Carta.cs`, `Jugador.cs`	500	–	–	–
`main.py`	223	10	–	–
`db.py`	–	21	–	–
`scriptuno.sql`	173	–	40	–

Nota: `git blame` cuenta quién escribió por última vez cada línea. Si alguien reescribe o reformatea una línea, queda a su nombre aunque la idea original fuera de otra persona. Por eso conviene leerlo junto con la lista de commits.

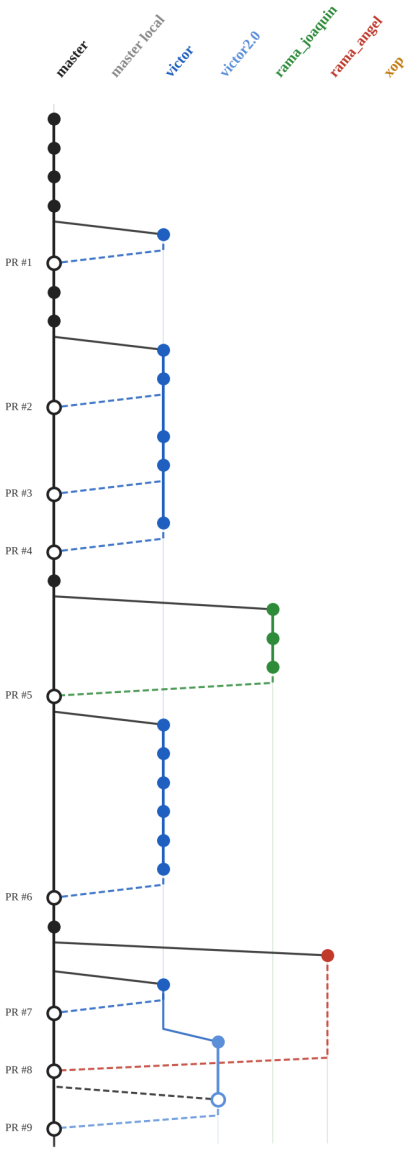
8.2 Diagrama de ramas

El diagrama muestra cómo se fueron creando y uniendo las ramas. Se dibujó a partir del historial real del repositorio (`git log` con los padres de cada commit). Cada columna de la izquierda es una rama y cada fila es un commit, en orden de fecha:

- Un punto relleno es un commit normal (un cambio en el código).
- Un círculo blanco es un merge (unión de dos ramas). Los que dicen "PR #n" son pull requests aceptados en GitHub.
- La línea continua une un commit con el anterior de la misma rama. La línea punteada indica de qué rama vienen los cambios que se unieron.
- Una línea que sale de `master` hacia otra columna significa que esa rama se creó (o se actualizó) a partir de `master` en ese momento.

La columna "master local" son tres commits que Ximena hizo en su copia local de `master` el 3 de octubre (`c85a81e`, `5c9b07f` y `526f137`). Se subieron a GitHub junto con `d908f85`, directamente a `master` y sin pull request, por eso no aparecen en ninguna rama con nombre.

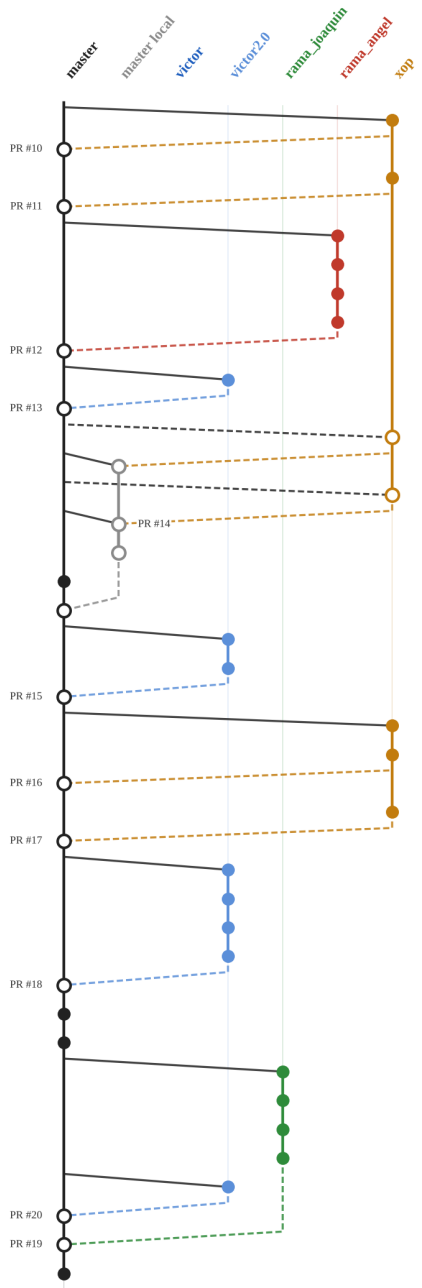
Del 21 de septiembre al 30 de septiembre de 2026



Fecha	Commit	Autor	Mensaje
21/09 11:56	`2ca8696`	Joaquin	Agregar .gitattributes y .gitignore.
21/09 11:56	`b5a4f53`	Joaquin	Agregar archivos de proyecto.
21/09 12:08	`d43c692`	Joaquin	Agrega nombre a la ventana y un icono al programa
22/09 11:43	`56edb42`	Joaquin	Cambio de color en la ventana
22/09 20:03	`1b6b360`	Victor	interfaz rudimentaria
22/09 20:23	`ded0b84`	Joaquin	PR desde victor
24/09 17:26	`0c69dd9`	Joaquin	Agrega paquete MySQL y jugador 4 en la interfaz
25/09 17:09	`4cb3969`	Ximena	Add script sql
26/09 14:10	`22745d3`	Victor	clases basicas, conexion a la bd y reparticion de …
26/09 14:28	`2215124`	Victor	se me olvido borrar mi clave
26/09 14:39	`4f48bba`	Victor	PR desde victor
26/09 19:53	`6f428ec`	Victor	robar y jugar cartas, todavia sin reglas
26/09 20:08	`ab876be`	Victor	Juego sin reglas funcional :O
26/09 21:47	`9722da7`	Victor	PR desde victor
27/09 13:10	`72ad9e9`	Victor	Turnos, reversa y +2, +4, aun sin validacion de ju…
27/09 13:14	`1d43ef1`	Victor	PR desde victor
27/09 17:25	`aebd27d`	Joaquin	Agrega lógica de carta +2
27/09 18:31	`299da74`	Joaquin	Agrega validación de cartas
27/09 19:54	`01abf64`	Joaquin	Agrega validación a carta +4 y cambio de color
27/09 20:06	`048039d`	Joaquin	Agrega ventana para elegir el color
27/09 20:17	`e4824ac`	Angel	PR desde rama_joaquin
27/09 20:43	`9987c92`	Victor	Valida la jugada antes de registrarla en el log
27/09 20:44	`0c11079`	Victor	Guarda el color elegido de los comodines en jugada…
27/09 20:44	`8883654`	Victor	Usa colorActual para el color en juego en lugar de…
27/09 20:45	`dd93642`	Victor	Permite jugar cartas con el mismo simbolo aunque c…
27/09 21:11	`32a434f`	Victor	boton pasar, color en label y mensaje de rechazo
27/09 21:20	`7fd8557`	Victor	Actualizacion del boton pasar
27/09 21:32	`bc85850`	Victor	PR desde victor
29/09 16:29	`e5a5652`	Joaquin	Agrega API para la conexión con la bd y librerias …
29/09 21:32	`3e6ef0f`	Angel	Agrega imagenes reales de las cartas en los Pictur…
30/09 00:13	`dfa1cbe`	Victor	API completa: endpoints de cartas, jugadores, part…
30/09 00:17	`4c6a4e7`	Victor	PR desde victor
30/09 14:38	`e3c6d12`	Victor	Base de datos actualizada e intslacion de paquetes…
30/09 21:43	`70f05a9`	Victor	PR desde rama_angel
30/09 21:56	`d8b6ebd`	Victor	Une master en victor2.0
30/09 22:23	`7793216`	Victor	PR desde victor2.0

Figura 2. Diagrama de ramas, del 21 al 30 de septiembre de 2026.

Del 2 de octubre al 10 de octubre de 2026



Fecha	Commit	Autor	Mensaje
02/10 12:39	`f7c8a11`	Ximena	Background Image
02/10 13:12	`176fe6e`	Ximena	PR desde xop
02/10 13:45	`2611b76`	Ximena	Background Image
02/10 13:51	`48abd34`	Ximena	PR desde xop
02/10 23:16	`e05be82`	Angel	Confirma ventana en pantalla completa y tamaño fijo
02/10 23:30	`0120a7f`	Angel	Quitar botonoes de jugar/robar y los comboBox, dej…
02/10 23:45	`459c293`	Angel	Agrega contenedores transparentes para las cartas
03/10 00:15	`82236fb`	Angel	Muestra las cartas como imagenes en la mano y perm…
03/10 10:51	`c504ce2`	Victor	PR desde rama_angel
03/10 11:39	`b2b8918`	Victor	refactorizacion de interfaz
03/10 11:40	`78d7e53`	Victor	PR desde victor2.0
03/10 20:34	`a1a08d8`	Ximena	Steal card corrected
03/10 20:44	`c85a81e`	Ximena	Steal card corrected
03/10 20:52	`234a0fd`	Ximena	Une master en xop
03/10 20:53	`5c9b07f`	Ximena	PR desde xop
03/10 21:12	`526f137`	Ximena	.
04/10 00:23	`cbcfd2a`	Ximena	Pick the new color window
04/10 00:29	`d908f85`	Ximena	Pick the new color window
04/10 00:56	`b0c00a6`	Victor	arreglo del problema merge conflict en el designer
04/10 00:57	`d532863`	Victor	Corrige robar sin partida y la ventana de color co…
04/10 01:01	`31dfd88`	Victor	PR desde victor2.0
05/10 23:11	`e3e761d`	Ximena	Start window added, still more changes to add
05/10 23:12	`631d0bd`	Ximena	Start window added, still more changes to add
05/10 23:14	`f569d79`	Ximena	PR desde xop
06/10 11:40	`312f7df`	Ximena	Method start game linked to the Start Window button
06/10 14:19	`b6f767b`	Ximena	PR desde xop
08/10 21:48	`6252950`	Victor	refactorizacion de la pantalla de inicio
08/10 21:56	`1865457`	Victor	actualizacion del boton pasar
08/10 22:08	`753ead2`	Victor	boton decir uno
08/10 22:17	`d4f1030`	Victor	panel de ganador
08/10 22:43	`98a53e6`	Victor	PR desde victor2.0
09/10 17:47	`a8ab47e`	Joaquin	Modifica label de inicio
09/10 17:55	`b7830da`	Joaquin	Agrega modificación al cursor sobre carta elegir c…
09/10 18:52	`62cd416`	Joaquin	Elimina boton sin uso cotidiano
09/10 20:20	`a1f83dc`	Joaquin	Elimina button9 sin utilizar en código
09/10 20:23	`b06abd7`	Joaquin	Elimina MessageBox que interrumpe flujo de juego c…
09/10 20:50	`bb4ea85`	Joaquin	Agrega efecto de prohibir jugar carta inválida
10/10 20:09	`c3bdf41`	Victor	version final
10/10 20:10	`de06b22`	Victor	PR desde victor2.0
10/10 20:21	`4c3b25a`	Victor	PR desde rama_joaquin
10/10 20:28	`9870aa0`	Angel	Delete CONTEXTO_PROYECTO_UNO.md

Figura 3. Diagrama de ramas, del 2 al 10 de octubre de 2026.

Lo que se ve en el diagrama:

- master es la rama principal. Joaquin hizo ahí los primeros cuatro commits (crear el proyecto). Después, casi todos los cambios llegaron por pull request.
- victor (del 22 al 30 de septiembre) y victor2.0 (del 30 de septiembre al 10 de octubre) son las ramas de Victor. Son las que tienen más pull requests (#1, #2, #3, #4, #6, #7, #9, #13, #15, #18 y #20).
- rama_joaquin tuvo dos etapas: el 27 de septiembre (lógica del +2, validación de cartas y ventana de color, PR #5) y el 9 de octubre (quitar botones y efecto rojo, PR #19, que fue el último en unirse).
- rama_angel tuvo dos etapas: imágenes de las cartas (PR #8) y paneles de cartas con doble clic (PR #12).
- xop es la rama de Ximena: imagen de fondo (PR #10 y #11), corrección de robar (PR #14) y pantalla de inicio (PR #16 y #17).
- En algunos puntos la rama se actualizó con `master` antes de hacer el pull request (por ejemplo `d8b6ebd` en victor2.0 y `234a0fd` en xop) para evitar conflictos.
- El último commit de `master` es `9870aa0` (Angel borra `CONTEXTO_PROYECTO_UNO.md`, 10 de octubre a las 20:28).

8.3 Todos los commits en orden

"PR #n" = un pull request aceptado en GitHub (un merge de una rama hacia `master`).

Semana 1: creación del proyecto (21–26 sep)

Fecha	Autor	Commit	Qué hizo
21-09 11:56	Joaquin	`2ca8696`	Agrega `.gitattributes` y `.gitignore`.
21-09 11:56	Joaquin	`b5a4f53`	Agrega los archivos del proyecto de Visual Studio.
21-09 12:08	Joaquin	`d43c692`	Nombre de la ventana e icono del programa.
22-09 11:43	Joaquin	`56edb42`	Cambio de color de la ventana.
22-09 20:03	Victor	`1b6b360`	Interfaz rudimentaria.
22-09 20:23	Joaquin	`ded0b84`	PR #1 (rama `victor`).
24-09 17:26	Joaquin	`0c69dd9`	Agrega paquete de MySQL y el jugador 4 en la interfaz.
25-09 17:09	Ximena	`4cb3969`	Agrega el script SQL (primera versión de la base, 121 líneas).
26-09 14:10	Victor	`22745d3`	Clases básicas (`Carta`, `Jugador`), conexión a la BD, `FormNombres` y reparto de cartas; actualiza el script SQL.
26-09 14:28	Victor	`2215124`	"Se me olvidó borrar mi clave": quita una contraseña que había quedado en el código.

Fecha	Autor	Commit	Qué hizo
26-09 14:39	Victor	`4f48bba`	PR #2.
26-09 19:53	Victor	`6f428ec`	Robar y jugar cartas, todavía sin reglas.
26-09 20:08	Victor	`ab876be`	Juego sin reglas funcional.
26-09 21:47	Victor	`9722da7`	PR #3.

Semana 2: reglas, API e imágenes (27 sep – 3 oct)

Fecha	Autor	Commit	Qué hizo
27-09 13:10	Victor	`72ad9e9`	Turnos, reversa, +2 y +4 (sin validación de jugadas).
27-09 13:14	Victor	`1d43ef1`	PR #4.
27-09 17:25	Joaquin	`aebd27d`	Lógica de la carta +2.
27-09 18:31	Joaquin	`299da74`	Validación de cartas (si se puede jugar o no).
27-09 19:54	Joaquin	`01abf64`	Validación del +4 y cambio de color.
27-09 20:06	Joaquin	`048039d`	Ventana para elegir el color.
27-09 20:17	Angel	`e4824ac`	PR #5 (rama `rama_joaquin`).
27-09 20:43	Victor	`9987c92`	Valida la jugada antes de registrarla en el log.
27-09 20:44	Victor	`0c11079`	Guarda el color elegido de los comodines en `jugada.color_elegido`.
27-09 20:44	Victor	`8883654`	Usa `colorActual` en lugar de modificar la carta.
27-09 20:45	Victor	`dd93642`	Permite jugar cartas con el mismo símbolo aunque cambie el color.
27-09 21:11	Victor	`32a434f`	Botón Pasar, color en la etiqueta y mensaje de jugada rechazada.
27-09 21:20	Victor	`7fd8557`	Ajustes al botón Pasar.
27-09 21:32	Victor	`bc85850`	PR #6.
29-09 16:29	Joaquin	`e5a5652`	Agrega la API para conectarse a la BD (`db.py`, `main.py`) y las librerías necesarias.
29-09 21:32	Angel	`3e6ef0f`	Imágenes reales de las cartas en los PictureBox (53 archivos).
30-09 00:13	Victor	`dfa1cbe`	API completa: endpoints de cartas, jugadores, partidas, jugadas, historial y ganadores.
30-09 00:17	Victor	`4c6a4e7`	PR #7.
30-09 14:38	Victor	`e3c6d12`	Base de datos actualizada; `BaseDatos.cs` pasa a HTTP/JSON con Newtonsoft.Json.
30-09 21:43	Victor	`70f05a9`	PR #8 (rama `rama_angel`).
30-09 21:56	Victor	`d8b6ebd`	Une `master` en `victor2.0`.

Fecha	Autor	Commit	Qué hizo
30-09 22:23	Victor	`7793216`	PR #9.
02-10 12:39	Ximena	`f7c8a11`	Imagen de fondo.
02-10 13:12	Ximena	`176fe6e`	PR #10.
02-10 13:45	Ximena	`2611b76`	Imagen de fondo (ajustes).
02-10 13:51	Ximena	`48abd34`	PR #11.
02-10 23:16	Angel	`e05be82`	Ventana en pantalla completa y tamaño fijo.
02-10 23:30	Angel	`0120a7f`	Quita los botones de jugar/robar y los ComboBox.
02-10 23:45	Angel	`459c293`	Contenedores transparentes para las cartas.
03-10 00:15	Angel	`82236fb`	Cartas como imágenes en la mano, doble clic para jugar y scroll.
03-10 10:51	Victor	`c504ce2`	PR #12 (rama `rama_angel`).
03-10 11:39	Victor	`b2b8918`	Refactorización de la interfaz (acomodo de la mesa).
03-10 11:40	Victor	`78d7e53`	PR #13.
03-10 20:34	Ximena	`a1a08d8`	Corrige robar carta.
03-10 20:44	Ximena	`c85a81e`	Corrige robar carta.
03-10 20:52	Ximena	`234a0fd`	Une `master` en `xop`.
03-10 20:53	Ximena	`5c9b07f`	PR #14.
03-10 21:12	Ximena	`526f137`	Commit con mensaje "." (sin descripción).

Semana 3: ventanas finales (4–10 oct)

Fecha	Autor	Commit	Qué hizo
04-10 00:23	Ximena	`cbcfd2a`	Ventana para elegir color con imágenes (las imágenes en el .resx).
04-10 00:29	Ximena	`d908f85`	Ventana de elegir color (ajustes).
04-10 00:56	Victor	`b0c00a6`	Arregla el conflicto de merge que quedó en el diseñador.
04-10 00:57	Victor	`d532863`	Corrige robar sin partida y la ventana de color cortada.
04-10 01:01	Victor	`31dfd88`	PR #15.
05-10 23:11	Ximena	`e3e761d`	Agrega la pantalla de inicio (primera versión).
05-10 23:12	Ximena	`631d0bd`	Pantalla de inicio (archivos del formulario y recursos).
05-10 23:14	Ximena	`f569d79`	PR #16.
06-10 11:40	Ximena	`312f7df`	Conecta el botón de la pantalla de inicio con empezar partida.
06-10 14:19	Ximena	`b6f767b`	PR #17.
08-10 21:48	Victor	`6252950`	Refactorización de la pantalla de inicio (`BotonUno`, estado de la API).

Fecha	Autor	Commit	Qué hizo
08-10 21:56	Victor	`1865457`	Actualización del botón Pasar.
08-10 22:08	Victor	`753ead2`	Botón ¡UNO! con cuenta regresiva y castigo.
08-10 22:17	Victor	`d4f1030`	Pantalla del ganador (`FormGanador`).
08-10 22:43	Victor	`98a53e6`	PR #18.
09-10 17:47	Joaquin	`a8ab47e`	Modifica la etiqueta de estado del inicio (ciclo que revisa la API).
09-10 17:55	Joaquin	`b7830da`	Cursor de mano en las cartas de elegir color.
09-10 18:52	Joaquin	`62cd416`	Elimina un botón sin uso.
09-10 20:20	Joaquin	`a1f83dc`	Elimina `button9` ("Empezar juego") y su método `button9_Click`.
09-10 20:23	Joaquin	`b06abd7`	Quita el `MessageBox` que mostraba el color elegido (interrumpía el juego).
09-10 20:50	Joaquin	`bb4ea85`	Efecto rojo al intentar jugar una carta inválida (`EfectoCartaInvalida`).
10-10 20:09	Victor	`c3bdf41`	Versión final: documento de contexto y ajustes en FormInicio.
10-10 20:10	Victor	`de06b22`	PR #20 (rama `victor2.0`).
10-10 20:21	Victor	`4c3b25a`	PR #19 (rama `rama_joaquin`).
10-10 20:28	Angel	`9870aa0`	Borra `CONTEXTO_PROYECTO_UNO.md` (último commit de `master`).

8.4 Problemas que se ven en el historial

- Contraseña en el código (`2215124`): la contraseña de MySQL estuvo escrita en un commit. Se quitó después y ahora vive en `.env` (ignorado por Git), pero sigue visible en el historial de los commits antiguos.
- Conflicto de merge en el diseñador (`b0c00a6`): un merge dejó marcas `<<<<<<<` dentro de `Form1.Designer.cs` y el proyecto no compilaba. Por eso el equipo decidió crear estilos y controles nuevos desde código (`AplicarEstilo`) y no desde el diseñador.
- Cambio de arquitectura (`e5a5652`, `dfa1cbe`, `e3c6d12`): al principio C# se conectaba a MySQL con MySqlConnector; después se cambió a la API en Python por requisito del profesor. El paquete `MySqlConnector` quedó en `packages.config` sin usarse.
- Commits subidos directo a master: `cbcfd2a` y `d908f85` (junto con los de la columna "master local") entraron a `master` sin pull request, a diferencia del resto del equipo.
- Pull requests fuera de orden: el PR #20 se unió antes que el #19 (el #19 se había abierto antes, pero se aceptó 11 minutos después).
- Mensajes poco descriptivos: hay commits con mensaje "." (`526f137`) o mensajes repetidos.

9. Guía de estudio

9.1 Dónde está cada cosa importante

Tema	Archivo	Líneas
Validar si una carta se puede jugar	Form1.cs	252–262
Efecto rojo de carta inválida	Form1.cs	305–344
Efectos de las cartas especiales	Form1.cs	388–413
Calcular el siguiente turno (módulo)	Form1.cs	418–422
Barajar (Fisher-Yates)	Form1.cs	511–520
Repartir 7 cartas	Form1.cs	526–536
Carta inicial solo de número	Form1.cs	544–555
Empezar partida (todo el flujo)	Form1.cs	189–242
Robar	Form1.cs	915–933 y 429–462
Decir UNO y castigo	Form1.cs	943–1003
Ganador	Form1.cs	286–287 y 464–489
Doble clic para jugar	Form1.cs	599
Imágenes de cartas y caché	Form1.cs	709–768
Enviar peticiones HTTP	BaseDatos.cs	161–198
Conversión JSON ↔ C#	BaseDatos.cs	226–250
Transacción al crear partida	main.py	134–158
Número de jugada consecutivo	main.py	181–184
Errores de MySQL → HTTP	main.py	40–46
Reglas de consistencia de cartas	scriptuno.sql	20–26
Reglas del log	scriptuno.sql	104–111
Vista de historial	scriptuno.sql	118–127
Revisar si la API está encendida	FormInicio.cs	134–157
Botón dibujado a mano	FormInicio.cs	191–259
Animación de confeti	FormGanador.cs	80–108

9.2 Preguntas que te pueden hacer (con respuesta)

- ¿Por qué hay una API en Python si el juego es en C#? Requisito del profesor: C# no debe tocar MySQL. C# manda JSON por HTTP a FastAPI y FastAPI es la única que habla con MySQL.
- ¿Dónde se guardan las manos de los jugadores? En memoria, en `Jugador.Mano` (Jugador.cs línea 13). La base solo guarda el log de movimientos; con él se puede reconstruir cualquier mano.
- ¿Cómo se sabe quién sigue? `Siguiente()` (Form1.cs líneas 418 a 422) con `direccion` (1 o −1) y módulo para dar la vuelta.
- ¿Cómo funciona la reversa? Invierte `direccion` y avanza un lugar en el nuevo sentido (líneas 392 a 395).
- ¿Qué pasa si la API falla a mitad de una jugada? Se registra en la base antes de cambiar el estado (líneas 272 a 280); si falla, la jugada no se hace y el juego queda igual.
- ¿Por qué una transacción en POST /partidas? Son 34 inserts (1 partida + 4 jugadores + 28 repartos + 1 carta inicial). Si uno falla, `rollback` deshace todo y no queda una partida incompleta.
- ¿Qué evita que se registre una jugada de alguien que no está en la partida? La llave foránea compuesta `fk_jugada_pj` (scriptuno.sql líneas 98 a 100).
- ¿Cómo se calcula num_jugada? Con `insert ... select coalesce(max(num_jugada), 0) + 1` en una sola consulta (main.py líneas 181 a 184).
- ¿Por qué `Task.Run` en BaseDatos? Para evitar el deadlock de esperar una tarea asíncrona desde el hilo de la ventana (líneas 176 a 177).
- ¿Para qué sirve el caché de imágenes? Para cargar cada PNG una sola vez; cargarlo en cada jugada gasta memoria y deja archivos bloqueados (línea 70, líneas 731 a 746).
- ¿Qué es Fisher-Yates? Un algoritmo para barajar donde cada orden posible es igual de probable; recorre de atrás hacia adelante intercambiando con una posición aleatoria anterior.
- ¿Cómo se cuentan las victorias en la vista? `sum(p.id_ganador = j.id_jugador)`: la comparación vale 1 o 0 y se suma.
- ¿Qué pasa si cierro el juego a mitad de partida? `Ventana_FormClosing` la termina sin ganador (`id_ganador = null`), queda como abandonada y no cuenta en el historial.
- ¿Qué pasa si intento jugar una carta que no se puede? No pasa nada en el juego: la carta se pinta de rojo 350 ms (`EfectoCartaInvalida`, líneas 305 a 344) y el turno sigue igual.
- ¿Por qué `colorActual` en vez de cambiar el color de la carta? Porque los comodines no tienen color en la base (el check lo prohíbe); el color elegido se guarda aparte y en

`jugada.color_elegido`.

9.3 Glosario rápido

Término	Significado
API	Programa que recibe peticiones y responde datos; aquí, por HTTP con JSON.
Endpoint	Una ruta de la API (ej. `POST /jugadores`).
JSON	Formato de texto para datos: `{"nombre": "Ana"}`.
FastAPI / uvicorn	Framework de Python para APIs / servidor que la ejecuta.
Pydantic	Valida que el JSON tenga la forma correcta.
DTO	Clase que solo sirve para transportar datos (ej. `CartaApi`).
Serializar / deserializar	Objeto → JSON / JSON → objeto.
Llave foránea (FK)	Columna que debe existir en otra tabla.
Check	Regla que MySQL revisa en cada fila.
Transacción	Varias operaciones que se guardan todas o ninguna (`commit` / `rollback`).
Vista	Consulta guardada que se usa como tabla.
Context manager	Objeto para `with` que abre y cierra un recurso automáticamente.
Modal	Ventana que bloquea a la de atrás hasta cerrarse (`ShowDialog`).
Evento	Algo que pasa (clic, tic del timer) y ejecuta una función suscrita.
Reflexión	Acceder a partes de una clase en tiempo de ejecución, incluso privadas.
Doble buffer	Dibujar en memoria antes de mostrar, para evitar parpadeo.
Pull request (PR)	Solicitud para unir una rama a `master` en GitHub.